

FACULTY
OF
INFORMATION
AND
COMMUNICATION TECHNOLOGY

NATIONAL DIPLOMA: COMPUTER SYSTEMS ENGINEERING

USER MANUAL – VERSION 2.0

SUBJECT NAME: Project Design 376

SUBJECT CODE: PD2316D

PROJECT TITLE: Smart Registration System for Exam Venue Access Based on Face Recognition

LECTURER NAME: Muwanguzi Mark Ntume and Chunling Du

STUDENTS NAME: TC Mance

STUDENTS NUMBER: 222138469

EMAIL ADDRESS: 222138469@tut4life.ac.za

CONTACT NUMBER: 063 392 8361

SUBMISSION DATE: 5 November 2026

TABLE OF CONTENTS

List of Figures and Tables....3
List of Figures....3
List of Tables....4
1. Scope....5
1.1 Identification....5
1.2 System overview....5
1.3 Document overview....6
2. Referenced Documents....7
3. Software Summary....7
3.1 Software application....7
3.2 Software inventory....8
3.3 Software environment....9
3.4 Software organisation and overview of operation....9
3.4.1 Logical components (user's point of view)....9
3.4.2 Performance characteristics....9
3.4.3 Relationship to other systems and people....10
3.4.4 Supervisory and security controls....10
3.5 Contingencies and alternate modes of operation....10
3.6 Security and privacy....11
3.7 Assistance and problem reporting....11
4. Access to the Software....12
4.1 First-time user....12
4.1.1 Access control....12
4.1.2 Installation and setup....12
4.2 Initiating procedure (exam day)....14
4.3 Description of inputs....15
4.3.1 Input conditions....15
4.3.2 Input formats....16
4.3.3 Composition rules....16
4.3.4 Input vocabulary....16
4.3.5 Sample inputs....17
4.4 Description of outputs....17
4.4.1 General description....17
4.4.2 Output formats at the door....17
4.4.3 Sample outputs....18
4.5 Recovery and error correction....19
4.6 Stopping and suspending work....19
5. Processing Reference Guide....20
5.1 Capabilities....20
5.2 Conventions....20
5.3 Processing procedures....20
5.3.1 Sign in and sign out....20
5.3.2 Manage venues (administrator)....21
5.3.3 Manage door units (administrator)....21
5.3.4 Enrol a student....22
5.3.5 Students and modules....24
5.3.6 Book an exam....25

5.3.7 Run the door unit....28
5.3.8 Handle a denied student or a "No seat" flag (manual override)....28
5.3.9 Accounts (administrator)....29
5.3.10 E-mail log....29
5.3.11 Replace a door PC (administrator)....30
5.4 Related processing (background tasks)....30
5.5 Data backup....30
5.6 Recovery from errors, malfunctions and emergencies....31
5.7 Messages....31
5.8 Quick-reference guide....31
6. Query Procedures....32
6.1 Database and data file format....33
6.2 Query capabilities....34
6.3 Query preparation....34
6.4 Control instructions....35
7. Notes....35
7.1 Acronyms and abbreviations....35
7.2 Known limitations....35
7.3 Document history....36
7.4 Summary....36
7.5 Figure sources....36
8. References....36
Appendix A: Hardware Guide....37
A.1 Parts list....37
A.2 Wiring....37
A.3 Power-up and shutdown....38
A.4 Mounting and positioning....38
A.5 Hardware test....38
A.6 Care and maintenance....38
Appendix B: Door Program Settings....38
Appendix C: Database Creation....39

List of Figures and Tables

List of Figures

Figure 1: System context: door unit, backend, database and SRS Console....6
Figure 2: UML use case diagram: who can do what....8
Figure 3: SRS Console sign-in screen....12
Figure 4: UML deployment diagram: where each part runs....13
Figure 5: Exam-day procedure....15
Figure 6: Student enrolment screen....16
Figure 7: UML state machine of the door unit with its LCD messages....17
Figure 8: Dashboard with live statistics, students recognised now and exam bookings....18
Figure 9: Troubleshooting at the door....19
Figure 10: SRS Console menu map....20
Figure 11: Venues and devices....21
Figure 12: Device page with the hidden device key and its enrolled students....22
Figure 13: UML sequence diagram: enrolling a student....23
Figure 14: Student list with search and filters....24

Figure 15: Modules....25
Figure 16: Exam bookings....26
Figure 17: Exam calendar....26
Figure 18: Exam sessions, once-per-session entry and automatic sign-out....27
Figure 19: UML sequence diagram: booking an exam, e-mails and automatic sign-out....27
Figure 20: UML activity diagram: what happens when a student walks up....28
Figure 21: Accounts (administrators only)....29
Figure 22: Email log....30
Figure 23: UML class diagram of the database (11 tables)....33
Figure 24: Door unit wiring....37

List of Tables

Table 1: System identification....5
Table 2: System parts....5
Table 3: Readers of this manual....6
Table 4: Referenced documents....7
Table 5: Software inventory....8
Table 6: Software environment....9
Table 7: Logical components....9
Table 8: Performance characteristics....10
Table 9: Operating modes....10
Table 10: What is stored and where....11
Table 11: Assistance....11
Table 12: Backend settings....14
Table 13: Exam-day steps....15
Table 14: Input conditions....15
Table 15: Enrolment fields....16
Table 16: Status words....16
Table 17: Outputs....17
Table 18: Door results....18
Table 19: Sample dashboard rows (demo data)....18
Table 20: Problems and what to do....19
Table 21: Conventions....20
Table 22: Background tasks....30
Table 23: Messages....31
Table 24: Main tables....34
Table 25: Acronyms and abbreviations....35
Table 26: Document history....36
Table 27: Door unit parts....37
Table 28: Wiring (Arduino Mega)....37
Table 29: config.ini and command-line options....38

1. Scope

1.1 Identification

Table 1: System identification

Item	Detail
System	Smart Registration System for Exam Venue Access Based on Face Recognition
Abbreviation	SRS
Web application name	SRS Console
Version / release	v2.0 / November 2026 (replaces v1.0 of 8 October 2026)
Software this manual covers	(1) PC door program (srs_pc_app.py) and Arduino door unit sketch (srs_door.ino) (2) Backend server (REST API) (3) Web application (SRS Console)
Operating platforms	SRS Console: any current desktop or tablet browser · Backend: Node.js 24 on Render · Database: PostgreSQL on Neon · Door program: Windows PC with Python 3 and OpenCV · Door unit: Arduino Mega 2560 (Uno/Nano also work)

1.2 System overview

Purpose. During exams, invigilators normally check attendance registers and student cards by eye. This disturbs students who are writing, and an impersonator can be missed. The SRS replaces the visual check with an automatic face check at the exam venue entrance and keeps a record of every attempt.

How it works in one paragraph. A door unit stands at the venue entrance: a Windows PC with a USB webcam and a small Arduino box with a distance sensor, an LCD screen and a buzzer. When a student stands 25–50 cm from the sensor and holds still for one second, the PC takes the picture and compares the face with the faces enrolled on that PC. A recognised student is let in once per exam session: the LCD shows ACCESS GRANTED and the PC window shows the student's seat number and module, or warns that the student has no seat in this session. An unknown face gets ACCESS DENIED, and the invigilator checks the student card by hand. Every decision appears on the live dashboard of the SRS Console.

Table 2: System parts

Part	What it is	Who uses it
Door unit	Windows PC + USB webcam running the PC door program; Arduino Mega with HC-SR04 distance sensor, 16×2 LCD and buzzer	Students stand in front of it; invigilators set it up and start it
Backend server	Node.js + Express REST API on Render with a PostgreSQL database on Neon	Used automatically by the door unit and the console
SRS Console	Browser-based web app (React) on Vercel	Invigilators and administrators

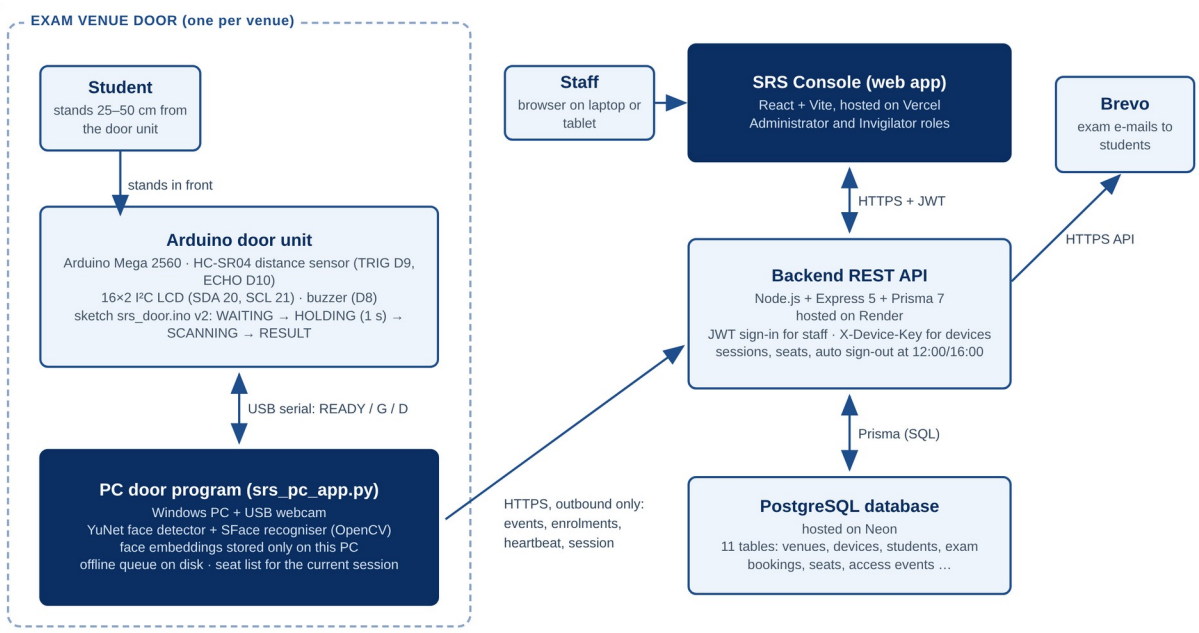


Figure 1: System context: door unit, backend, database and SRS Console

How to read this figure. Boxes are parts of the system; arrows show who talks to whom. The door unit only sends requests out to the backend (it never receives calls), and the SRS Console only talks to the backend, never to the door unit.

Development, sponsor and users. The system was designed and built by the student named on the cover page as the final project of Project Design 376 (PD2316D) at the Tshwane University of Technology (TUT). The intended users are TUT exam invigilators and the administrator who runs the system. The system supports many venues and door units. It does not connect to the university's student information system, so students, modules and exams are entered in the SRS Console.

History. The approved proposal (R1) defined the problem; the design document (R2) fixed a three-part architecture with an ESP32-CAM door device; manual v1.0 described that design. Because of hardware problems with the ESP32-CAM, the door device was rebuilt as a PC + webcam + Arduino door unit. This manual (v2.0) documents that final system.

1.3 Document overview

This manual explains how to set up, operate, monitor and recover the SRS. It is written for two groups of readers.

Table 3: Readers of this manual

Reader	What they do	Read these parts
Invigilator (non-technical)	Signs in, enrols students, starts the door unit, watches the dashboard, handles denied students	Sections 3.1, 4.1, 4.2 to 4.6, 5.3 and the Quick-Start Card in 5.8
Administrator (technical)	Deploys the backend and console, sets up door PCs and Arduinos, manages venues, devices, accounts and backups	Sections 4.1.2 Part B, 3.6, 5.5, 6 and Appendices A to C

Security and privacy of this manual. This manual contains no passwords, device keys or database connection strings. Screenshots use demonstration data only. See Section 3.6 for the privacy rules of the system itself.

2. Referenced Documents

Table 4: Referenced documents

No.	Title	Version / date	Source
R1	Project Proposal: Smart Registration System for Exam Venue Access	v5, 20 Aug 2026	Student, via Brightspace
R2	Design Documentation: Smart Registration System	Sep 2026	Student, via Brightspace
R3	User Manual v1.0	8 Oct 2026	Student
R4	Final Project Report: Smart Registration System	Nov 2026	Student
R5	Project Design 376 / Industry Exposure Student Course Guide (PD2316D)	Jul 2025	Du, C. and Muwanguzi Ntume, M., TUT
R6	Device API (DEVICE_API.md) and READMEs in the source repository	Current	github.com/TebogoClifford/ Smart_Registration_System
R7	OpenCV Zoo: YuNet face detector and SFace face recogniser	Current	https://github.com/opencv/ opencv_zoo
R8	Arduino Mega 2560 Rev3 documentation	Current	https://docs.arduino.cc/hardware/ mega-2560/
R9	HC-SR04 ultrasonic ranging module datasheet	–	Module vendor (ElecFreaks)
R10	Protection of Personal Information Act 4 of 2013 (POPIA)	2013	Government of South Africa
R11	Information Regulator Guidance Note on Processing of Special Personal Information	28 Jun 2021	https://inforegulator.org.za
R12	PostgreSQL 16 documentation; Prisma ORM documentation	Current	postgresql.org/docs · prisma.io/docs

3. Software Summary

3.1 Software application

Intended use. To verify, at the entrance of an exam venue, that a person presenting for an exam is an enrolled student, to show their seat for the session, and to keep an auditable record of every attempt.

Capabilities.

- Automatic face check, taken only when the student holds still 25–50 cm from the door unit for one second.
- Each student is let in once per exam session; a second scan shows ALREADY INSIDE.
- Seat number and module shown on the PC window; students with no seat in the session are flagged "No seat" on the dashboard.
- Automatic sign-out of everyone at 12:00 and 16:00, when the morning and afternoon sessions end.
- Live dashboard of all decisions, with an Override button that records manual decisions, and CSV export.
- Enrolment of a student's face from the SRS Console, carried out at the door unit.
- Modules, exam hall bookings with automatic seat allocation within the venue capacity, and an exam calendar.
- E-mails to students when an exam is booked, moved or cancelled, with an e-mail log.
- Staff accounts with two roles (administrator, invigilator) and secure sign-in.

- Keeps working when the internet drops: decisions continue and are uploaded later, never twice.

Expected benefits. Fewer interruptions to students during the exam, less reliance on an invigilator's eyesight, seat information at the door, and a reviewable record of who entered and when.

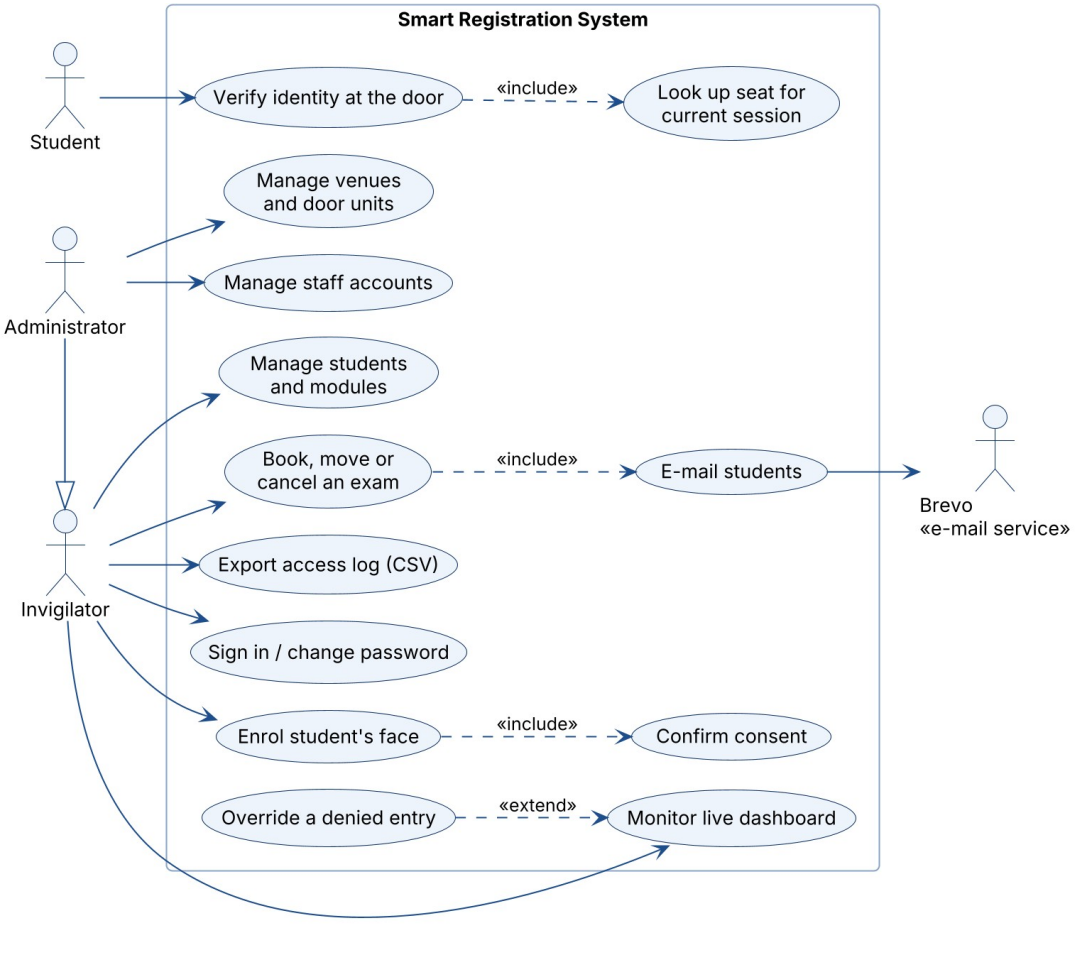


Figure 2: UML use case diagram: who can do what

How to read this figure. Stick figures are actors (people or outside systems); ellipses inside the box are the things the system lets them do. The hollow-triangle arrow means an Administrator can do everything an Invigilator can. «include» marks a step that always happens as part of another; «extend» marks an optional step.

3.2 Software inventory

Table 5: Software inventory

Item	Where it runs or lives	Sensitive?	Needed to continue in an emergency?
PC door program (Python, OpenCV)	Door PC, folder pc_device	No	Yes: the source and the two model files
Face models (YuNet, SFace, ~39 MB)	Door PC, pc_device/models	No	Yes: re-download with download_models.py
Face embeddings + enrolment photos	Door PC, pc_device/data/embeddings and data/photos	Yes (biometric)	Yes, or re-enrol the students
Offline event queue	Door PC,	Yes (student IDs)	Uploaded automatically

Item	Where it runs or lives	Sensitive?	Needed to continue in an emergency?
	pc_device/data/event_queue.json		when online
Arduino sketch (srs_door.ino v3)	Arduino flash memory	No	Yes: re-upload from the source
Backend (Node.js, Express, Prisma)	Render web service	No (code); settings are secret	Yes
Database (PostgreSQL)	Neon	Yes (students, e-mails, access log, photos)	Yes: restore from backup (5.5)
SRS Console (React)	Vercel	No	Yes
Secrets	Render settings; config.ini on each door PC	Yes	Yes: keep a secure offline copy

Definitions. An **embedding** is a list of 128 numbers that represents a face so two faces can be compared. A **device key** is the secret password a door unit sends with every request so the backend knows it is genuine.

Continuing in an emergency. If any part is unavailable, exam entry continues by manual card checking. No exam depends on the SRS working.

3.3 Software environment

Table 6: Software environment

Requirement	Detail
Invigilator computer	A laptop, desktop or tablet with a current web browser and internet. Nothing to install.
Door PC	Windows 10 or 11 PC or laptop; Python 3 with the packages in pc_device/requirements.txt (opencv-python, numpy, requests, pyserial); free USB ports for the webcam and Arduino.
Webcam	Any USB webcam (720p or better). With camera = auto the highest-numbered camera is used, normally the USB webcam.
Door unit	Arduino Mega 2560 (or Uno/Nano), HC-SR04 sensor, 16×2 LCD with I²C backpack, passive or active buzzer (Appendix A).
Power	Mains power for the PC; the Arduino is powered from the PC's USB port. No batteries.
Network	Internet access from the door PC (outbound HTTPS only, any network). If Wi-Fi drops, events are queued and sent later.
Hosting (administrator)	Render account (backend), Neon account (database), Vercel account (console), optional Brevo account (e-mails).
Manual procedures	Student card check for denied students and students with no seat; student consent before enrolment.

3.4 Software organisation and overview of operation

3.4.1 Logical components (user's point of view)

Table 7: Logical components

Component	What the user sees
Door unit – LCD and buzzer	Instructions (come closer, hold still) and the result with a beep pattern (4.4.2)
Door unit – PC window	Camera view, result banner with name, seat and module, seat list for the session, enrolment in progress, Hardware panel
SRS Console	Dashboard, Venues & Devices, Students (list, enrolment, modules), Exam (bookings, calendar, e-mail log), Accounts, My account
Records	Venues, devices, students, modules, bookings, seats, access events, overrides, e-mails and staff accounts in one database

3.4.2 Performance characteristics

Values come from the program settings and from the benchmark in the final report (R4).

Table 8: Performance characteristics

Characteristic	Value
Capture distance window	25–50 cm, held still for 1 second
Recognition time (detect + embed + compare)	≈ 16 ms per 640×480 frame on a 2.1 GHz processor
Frames that must agree before a decision	3 in a row
Recognition accuracy (public LFW benchmark, threshold 0.40)	98.6%
False Acceptance Rate (stranger accepted)	0.03% (1:1); about 1.8% when compared with 100 enrolled faces
False Rejection Rate (enrolled student refused)	2.8%
Same student logged again at most every	15 s without the Arduino (60 s when already inside or session ended); with the Arduino every scan is logged
Enrolment	5 shots averaged; fails after 60 s without a good face
Door program checks for enrolments	every 3 s; session list refreshed every 2 minutes and at 12:00 / 16:00
Dashboard refresh	every 15 s; "Recognised now" panel every 3 s
Device shown Online	if it was heard from in the last 120 s
Offline queue upload	batches of up to 100 events
Factors that reduce accuracy	strong light behind the student, caps, sunglasses or masks, wrong distance, a face enrolled on another door unit

Polling means a program repeatedly asks the server "anything new?" at a fixed interval. It was chosen over pushed updates because it is simple and works through any router or firewall.

3.4.3 Relationship to other systems and people

The SRS is not connected to the university's student information system; students, modules and bookings are entered in the SRS Console. The invigilator remains responsible for the final decision at the door. The SRS supports that decision; it does not replace it.

3.4.4 Supervisory and security controls

- Staff sign in with e-mail and password. Accounts are created only by an administrator.
- New accounts and reset passwords get a temporary password that must be changed at first sign-in (at least 8 characters).
- After 5 failed sign-ins the account is blocked for 15 minutes. A sign-in lasts 12 hours.
- Administrators manage venues, devices, device keys and accounts. Invigilators cannot see device keys or the Accounts page.
- Each door unit sends its secret device key with every request; the backend refuses unknown devices.
- Only the administrator can reach the database and hosting settings.

3.5 Contingencies and alternate modes of operation

Table 9: Operating modes

Mode	When it happens	What still works	What to do
Normal	Door unit, internet and backend available	Everything	Follow Section 5
Offline door unit	Internet drops at the venue	Face checks continue (faces are on the PC); events queued on disk	Keep admitting; the dashboard catches up when online
Backend asleep	First request after a	The door unit retries; the	Wait a minute; refresh

Mode	When it happens	What still works	What to do
	quiet period (free hosting)	console may take up to 60 s to load	
No Arduino	Arduino unplugged or not connected	The PC scans any face that is big enough on camera; no LCD or beeps	Reconnect in the Hardware panel (5.3.7)
Door unit fault	Camera error, PC crash	Nothing automatic	Switch to manual card checks; see 4.5
Emergency evacuation	Fire or alarm	n/a	Leave the venue first; equipment can be replaced

3.6 Security and privacy

The data is biometric. A face photo and the embedding made from it are biometric information, which section 26 of POPIA treats as special personal information. It may only be processed under specific conditions such as the person's consent (R10, R11). The Enrolment screen therefore requires the tick box "The student has been told why their face is captured and agrees to it" before every enrolment. This manual is not legal advice; the institution's Information Officer should confirm the lawful basis and retention period before use on real students.

Table 10: What is stored and where

Data	Location	Notes
Face embedding	Door PC only (data/embeddings)	Never sent to the backend
One enrolment photo per student	Door PC and database	Shown on the console so staff can check who was enrolled
Student number, name, e-mail, course, modules	Database	Entered in the SRS Console
Access events (time, door unit, student, decision)	Database	Student is empty for unrecognised faces
Overrides (who, why)	Database	Recorded when an invigilator lets in a denied student
Video	Nowhere	The door unit never streams or stores video

Rules for users.

- WARNING: Do not share your sign-in details, and do not leave the console open on an unattended computer.
- Do not photograph or copy the dashboard, enrolment photos, CSV exports or database backups except for exam administration.
- Device keys are secrets: never paste them into chats or e-mails.
- Delete exports and backups when the retention period ends (at the end of the year).

3.7 Assistance and problem reporting

Table 11: Assistance

Problem	Contact	Details
Using the console, accounts, door unit faults	System administrator	Admin Test admin@srs.dev
Academic or project queries	Mr M. Muwanguzi Ntume (Soshanguve South, room 19-207, 012 382 9693); Prof C. Du (Soshanguve South, room 19-219, 012 382 9943, duc@tut.ac.za)	Book consultations at least three days ahead

When reporting a problem, give the date and time, the venue, what you were doing, what you saw on the screen or LCD, and (for the console) a screenshot with student details

hidden.

4. Access to the Software

4.1 First-time user

4.1.1 Access control

- **Obtaining an account.** An administrator creates your account on the Accounts page and gives you a temporary password. Invigilators cannot register themselves.
- **Web address.** The SRS Console is at https://smart-registration-system-silk.vercel.app/
- **First sign-in.** You must choose a new password of at least 8 characters before you can continue.
- **Changing your password.** My account → Change password. **Forgotten password:** ask an administrator to reset it (Accounts → Reset password).
- **Security reminders.** Sign out when you leave the computer. Treat every screenshot, export and printout as confidential.

4.1.2 Installation and setup

Part A: Invigilator (nothing to install). Open a current web browser, type the web address from 4.1.1 and sign in (Figure 3). If the page takes up to a minute the first time, wait: hosting that has been idle needs to wake up.

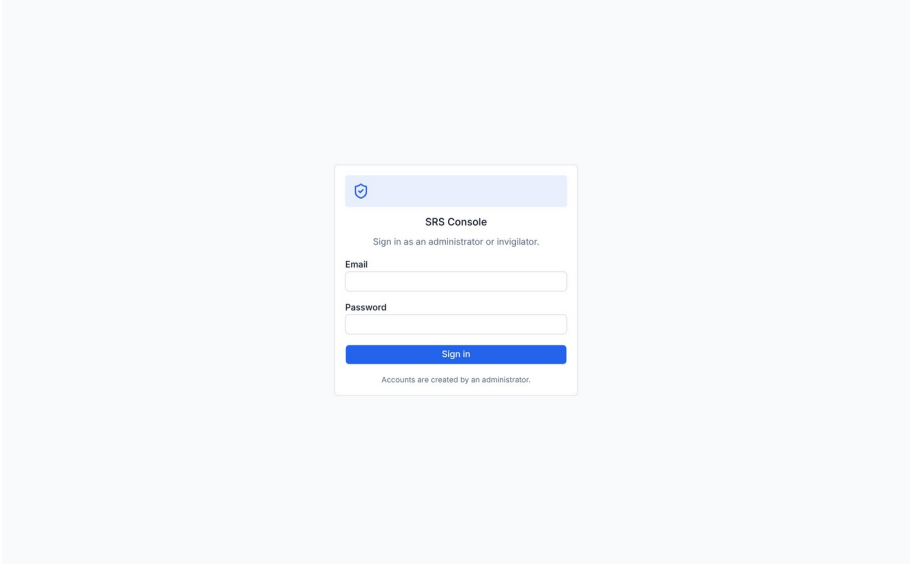


Figure 3: SRS Console sign-in screen

Part B: Administrator (one-time setup). Figure 4 shows where each piece of software is installed. Each step below lists the commands, with what each one does.

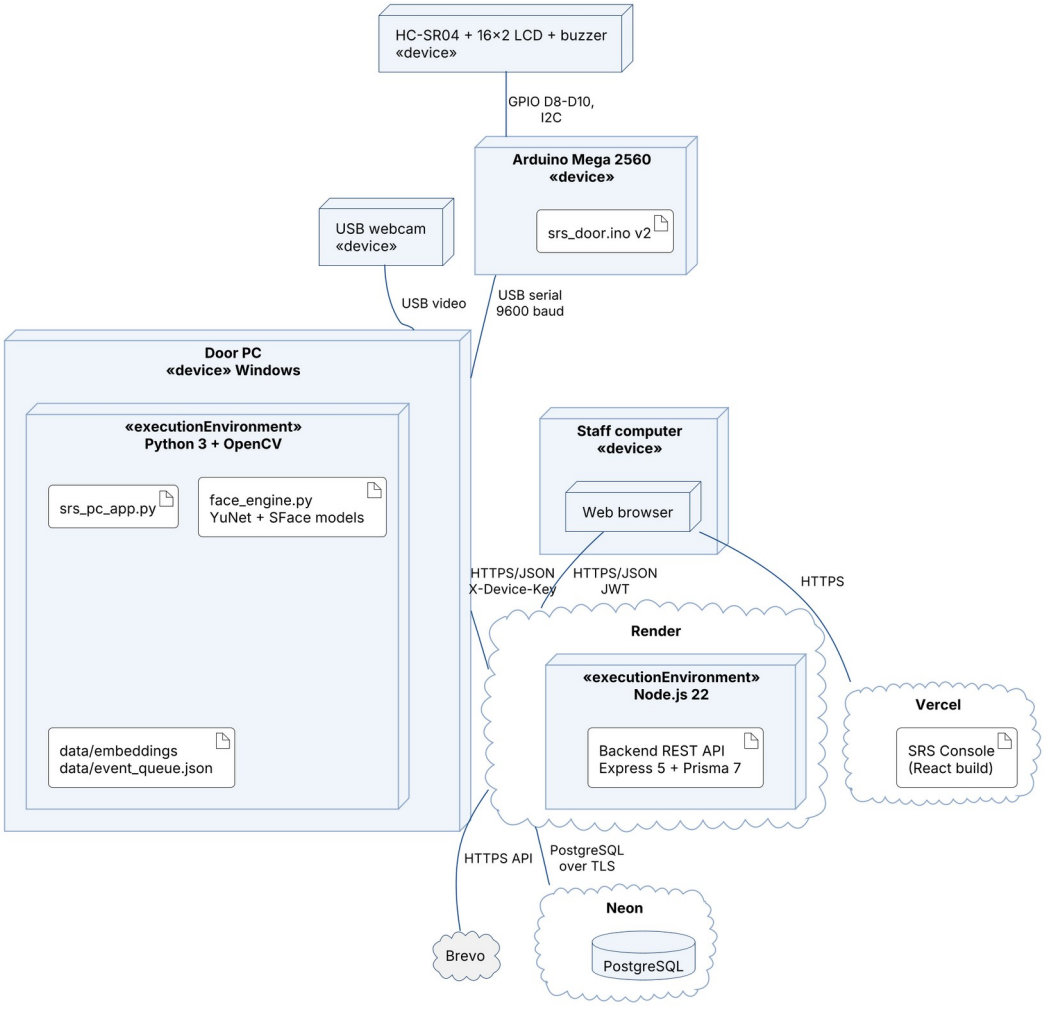


Figure 4: UML deployment diagram: where each part runs

How to read this figure. 3-D boxes are machines or cloud services (nodes); the document shapes inside are the programs and files installed there (artifacts); the lines are the connections, labelled with how they talk (USB, HTTPS, TLS).

B1. Database (Neon). Create a Neon project and copy its connection string (keep it secret). The tables are created automatically by the backend in step B2; no SQL script has to be run.

B2. Backend (Render).

```
# download the project source code
git clone https://github.com/TebogoClifford/Smart_Registration_System.git
# go into the backend folder
cd Smart_Registration_System/Backend
# install the libraries the backend needs (also generates the Prisma database client)
npm install
# start locally: applies the database migrations (creates the tables), then starts on port 3000
npm start
```

Settings (environment variables) for the backend, in a `.env` file locally or in the Render dashboard:

Table 12: Backend settings

Setting	Meaning
DATABASE_URL	Neon connection string (secret)
JWT_SECRET	At least 32 random characters used to sign sign-in tokens (secret)
ADMIN_EMAIL, ADMIN_PASSWORD	The first administrator account, created only when no accounts exist (password 8+ characters)
CORS_ORIGIN	The console's web address, e.g. https://smart-registration-system-silk.vercel.app
BREVO_API_KEY, EMAIL_FROM, EMAIL_MODE	Optional: e-mails to students. Without a key, e-mails are only recorded in the Email log
EXAM_TIMEZONE	Optional, default Africa/Johannesburg

On Render, create a **Web Service** from the repository with root directory `Backend`, build command `npm install` and start command `npm start`, and add the settings above.

B3. SRS Console (Vercel). Import the repository in Vercel (the root `vercel.json` builds the `web_app` folder) and set `VITE_API_BASE_URL` to the backend address, e.g. `https://smart-registration-system.onrender.com`.

B4. Door PC (once per door unit).

```
# go into the door program folder
cd pc_device
# install OpenCV, numpy, requests and pyserial
pip install -r requirements.txt
# download the two face models (~39 MB, checked against known SHA-256 values)
python download_models.py
# make your own settings file, then fill in url, device_id and device_key (Appendix B)
copy config.example.ini config.ini
# test the server address, device key and models without opening the camera
python srs_pc_app.py --check
# open the door window
python srs_pc_app.py
```

B5. Arduino door unit. Wire the parts as in Appendix A. In the Arduino IDE install the library "LiquidCrystal I2C" (Frank de Brabander), open `pc_device/arduino/srs_door/srs_door.ino`, choose the board (Arduino Mega or Mega 2560) and port, and upload. Close the IDE's Serial Monitor afterwards (only one program can use the port).

B6. Register the venue and door unit in the console (5.3.2 and 5.3.3), then copy the device id and key into `config.ini` on the door PC.

4.2 Initiating procedure (exam day)

Figure 5 shows the exam-day procedure; Table 13 gives the steps with the expected result of each.

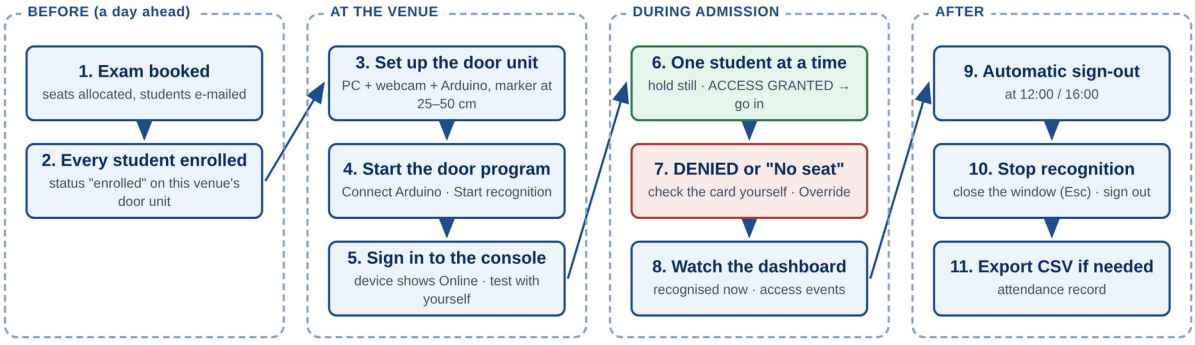


Figure 5: Exam-day procedure

How to read this figure. Read the four groups from left to right (before, at the venue, during admission, after) and the numbered boxes from top to bottom inside each group.

Table 13: Exam-day steps

Step	Action	Expected result
1	At least a day before: check the exam is booked (Exam → Exam bookings)	Booking listed; e-mails in the Email log (sent once e-mail is set up)
2	Check every student is enrolled on this venue's door unit (Students → Student list, filter by device)	Status "enrolled"
3	Set up the door unit: PC on mains, webcam at face height, Arduino connected, floor marker 25–50 cm from the sensor	LCD shows "NOT CONNECTED / Start the PC app" (the door program is not running yet)
4	Start the door program; in the Hardware panel press Connect; press ▶️ Start recognition	LCD: "Smart Register / Step up to scan"; banner: "Stand at the marker and hold still for 1 second"
5	Sign in to the SRS Console; open Venues & Devices	The door unit shows Online
6	Test with yourself or a known enrolled student	ACCESS GRANTED on the LCD; a row on the dashboard
7	Admit students one at a time	One dashboard row per attempt
8	For ACCESS DENIED or a "No seat" flag, check the student card yourself (5.3.8)	Student admitted or refused by you
9	When admission closes, stop recognition and close the door program (Esc); sign out of the console	Window closed; sign-in page shown

4.3 Description of inputs

4.3.1 Input conditions

Table 14: Input conditions

Input	Reason	Frequency	Origin	Related inputs
Sign-in details	Prove you are authorised staff	Each session	Staff member	None
Venue name and capacity	Group door units; limit seats	Once per venue	Administrator	None
Device name and venue	Register a door unit	Once per door unit	Administrator	Venue must exist
Student details	Create the student record	Once per student	Invigilator or admin	Venue and device
Module and registrations	Know who writes which exam	Once per module	Staff	Students exist
Exam booking	Book a venue, date and session	Once per exam	Staff	Module, venue capacity
Face capture (enrolment)	Teach the door unit a face	Once per student	Student at the door unit	Student saved first
Face capture	Decide at the door	Every entry	Student at the	Student enrolled on

Input	Reason	Frequency	Origin	Related inputs
(verification)			door unit	this unit

4.3.2 Input formats

Enrolment screen (Students → Enrolment, Figure 6):

Table 15: Enrolment fields

Field	What to enter	Limit
Student Number	As printed on the student card	20 characters
Full Name	As shown on the student card	100 characters
Email	The student's e-mail; exam bookings, changes and cancellations are e-mailed here	255 characters
Course (optional)	Choose from the list	n/a
Venue, Device	The venue and the door unit where the student will be checked	n/a
Consent tick box	"The student has been told why their face is captured and agrees to it"	Required

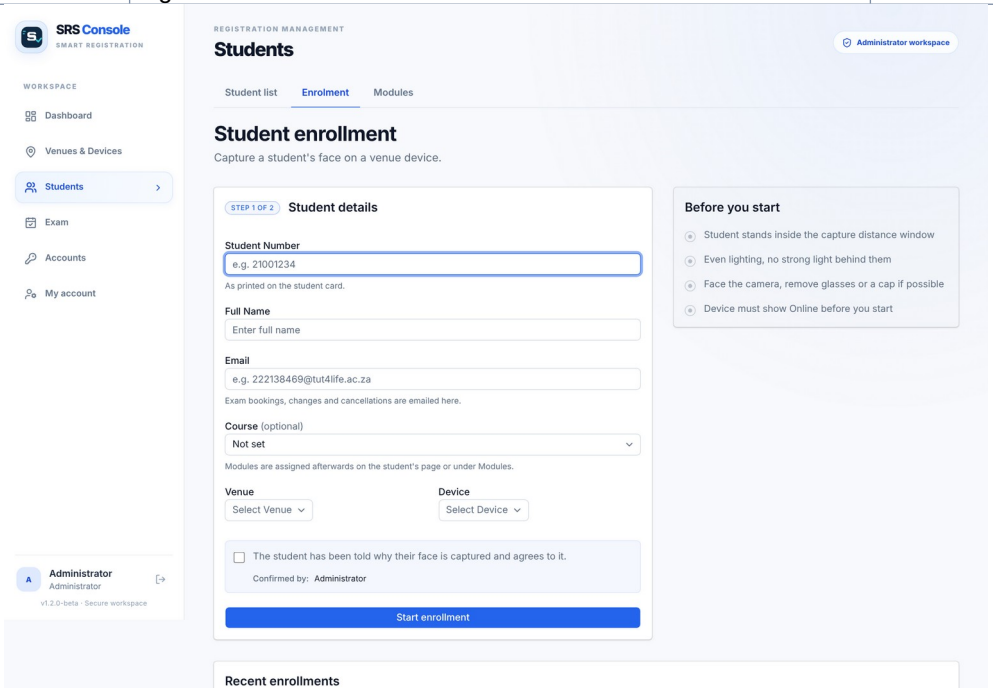


Figure 6: Student enrolment screen

4.3.3 Composition rules

- Student number: letters and digits, no spaces. E-mail: a valid address.
- Capture: one person in front of the camera, face uncovered (no cap peak, mask or sunglasses), looking at the lens, standing at the marker.
- Enrol in lighting similar to the venue entrance, without strong light behind the student.

4.3.4 Input vocabulary

Table 16: Status words

Field	Allowed values	Meaning
Enrolment status	pending, request expired, enrolled, failed	Waiting for the door unit · no door unit collected the request within 10 minutes (press Re-enrol) · face stored · no good capture within 60 s
Device status	Online, Offline	Heard from in the last 120 s, or not
Decision	granted, denied, error, already,	Let in · unknown face · capture failed · already

Field	Allowed values	Meaning
	ended, signed_out	inside this session · session over · automatic sign-out
Exam session (slot)	09:00, 13:00	Morning (ends 12:00) · afternoon (ends 16:00)
Role	admin, invigilator	Administrator · invigilator

4.3.5 Sample inputs

```
Student Number: 220100037        (sample data only)
Full Name:      Lerato Sithole
Email:          220100037@tut4life.ac.za
Venue:          Demo Main Hall
Device:         Demo Main Hall Camera
```

4.4 Description of outputs

4.4.1 General description

Table 17: Outputs

Output	Why	Frequency	Where
LCD text + beep	Tell the student the result at once, without waiting for the network	Every scan	Door unit
PC result banner	Name, student number, seat and module for the invigilator	Every scan	Door PC window
Dashboard rows and "Recognised now"	Show every attempt and flag students with no seat	Every scan	SRS Console
Enrolment confirmation	Confirm a student is ready	Once per enrolment	SRS Console
E-mail to student	Exam booked, moved or cancelled	Per booking change	Student's inbox; Email log
CSV export	Attendance record	On demand	Dashboard → Export CSV

4.4.2 Output formats at the door

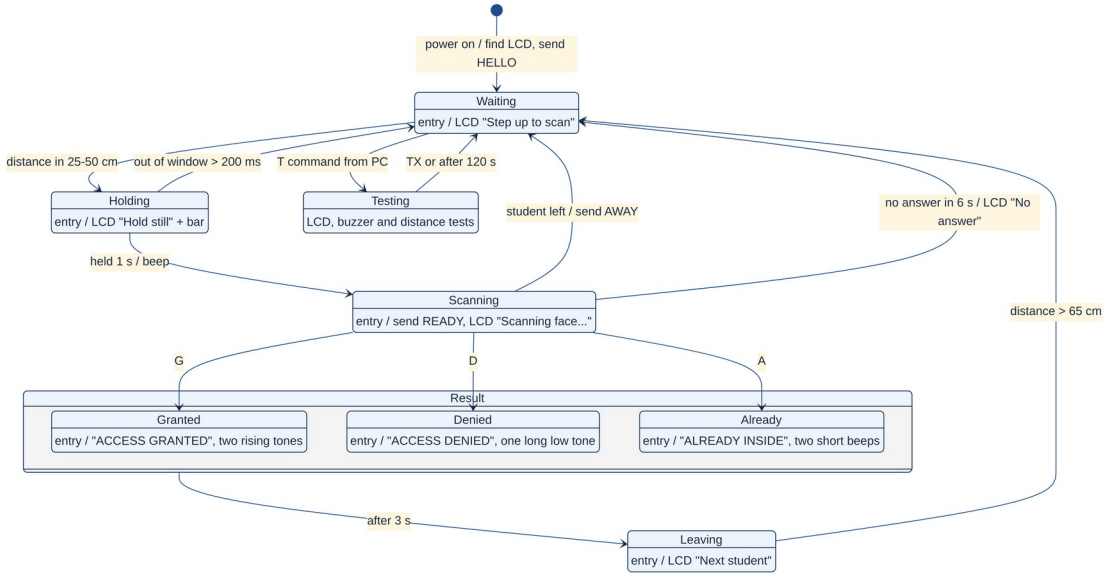


Figure 7: UML state machine of the door unit with its LCD messages

How to read this figure. This is a UML state machine. The black dot is power-on. Each

rounded box is a state; "entry /" is what the unit does when it enters that state (the exact LCD text and beep). Each arrow is labelled "event / action": what makes the unit move on, and what it does as it moves. The Result box groups the three possible results.

Table 18: Door results

Result	LCD	Buzzer	PC banner	Meaning
Granted	ACCESS GRANTED / Welcome, go in	Two rising tones	RECOGNISED name · seat N · module (or "no exam seat in this session")	Admit; check the card if there is no seat
Denied	ACCESS DENIED / See invigilator	One long low tone	NOT RECOGNISED – face is not enrolled	Check the student card manually
Already inside	ALREADY INSIDE / Entry recorded	Two short beeps	ALREADY INSIDE name	Was let in earlier this session
Session ended	ACCESS DENIED / See invigilator	One long low tone	SESSION ENDED name · signed out, not let in	Their session is over
No answer	No answer / Please try again	–	–	Student left, or the PC did not answer within 6 s

4.4.3 Sample outputs

Table 19: Sample dashboard rows (demo data)

Time	Device	Student	Decision
08:52:14	Demo Main Hall Camera	220100037 Lerato Sithole	Granted
08:52:41	Demo Main Hall Camera	–	Denied (Override available)
12:00:00	Demo Main Hall Camera	220100037 Lerato Sithole	Signed out (session ended)

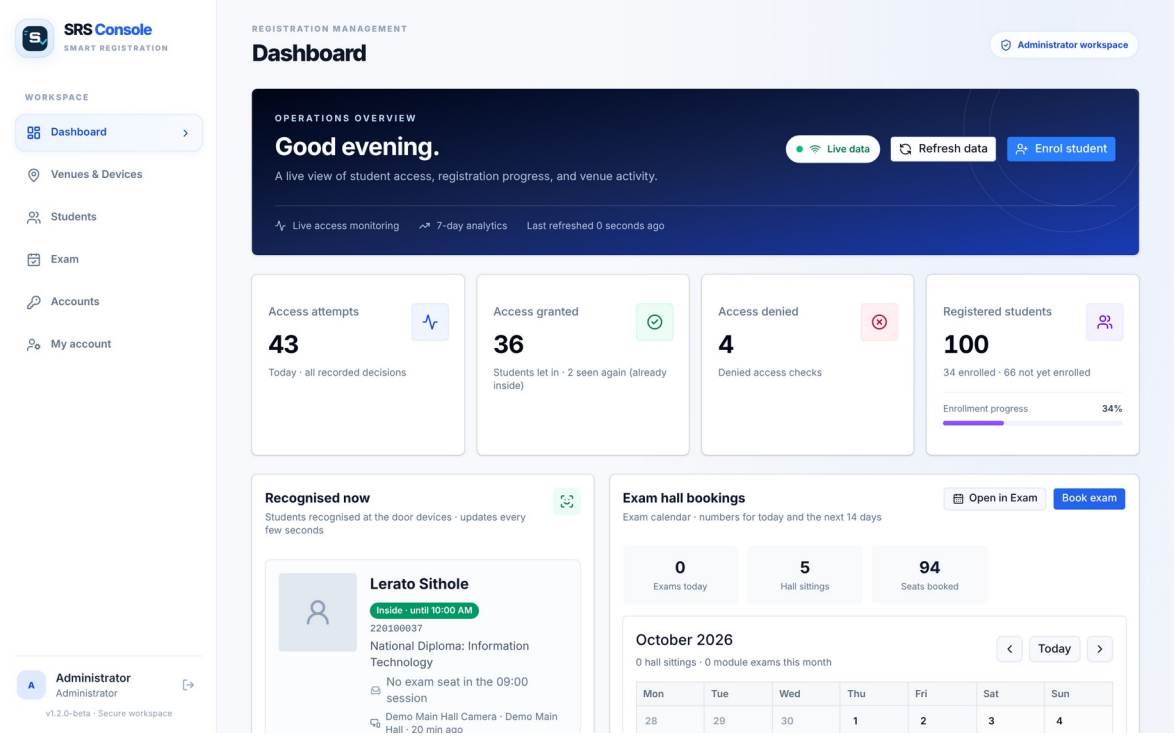


Figure 8: Dashboard with live statistics, students recognised now and exam bookings

4.5 Recovery and error correction

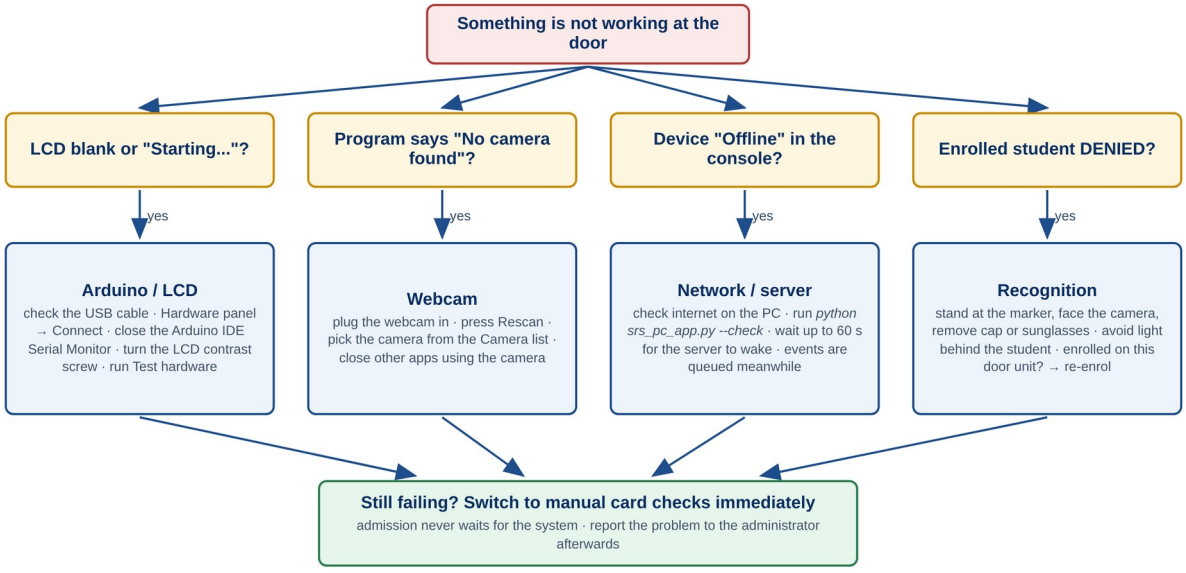


Figure 9: Troubleshooting at the door

How to read this figure. Start at the red box, follow the question that matches what you see, and try the checks in that column. If the problem remains, go to the green box: switch to manual checks.

Table 20: Problems and what to do

Situation	Meaning	What to do
Sign-in fails	Wrong details, or blocked after 5 attempts	Retype; wait 15 minutes; ask an administrator to reset the password
Signed out unexpectedly	The 12-hour sign-in ran out	Sign in again
Door unit shows Offline	Not heard from for 120 s	Check the PC is on, the program is running and the internet works; run `--check`
LCD shows NOT CONNECTED	The door program is closed or not connected to the Arduino	Start python srs_pc_app.py and press Connect in the Hardware panel
Enrolment stays "Capturing…"	The door unit has not captured yet	Check recognition window is open, the student is at the marker and only one face is in view
Enrolment "Failed"	No good face within 60 s	Improve lighting, remove face coverings, press Try again / Re-enrol
Enrolled student denied	Poor capture, light behind, or enrolled on another door unit	Re-position and try again; re-enrol on this door unit; admit by manual check
"No camera found"	Webcam missing or used by another app	Plug it in, press Rescan, pick it from the Camera list
LCD blank	Contrast, wiring or LCD not found	Turn the contrast screw; run Test hardware (5.3.7)
Event missing from the dashboard	Door unit offline or page not refreshed	Wait 15 s or refresh; queued events appear when the PC is online

Restart and continuity. If anything stops working during admission, switch to manual card checks immediately and fix the SRS later. Entry to the exam must never wait for the system.

4.6 Stopping and suspending work

- **Pause admission** (e.g. a break): press ■ Stop recognition; the window stays open.
- **End of admission:** close the door window (or press Esc), then sign out of the console. Events still in the queue are sent the next time the program runs.

- **Check it stopped normally:** the console returns to the sign-in page; after 120 s the door unit shows Offline.
- Unplug the Arduino and webcam only after the program is closed.

5. Processing Reference Guide

5.1 Capabilities

Figure 10 shows the SRS Console's menu. The first time, work from left to right (venues and devices, students, exams); on exam days use the Dashboard.

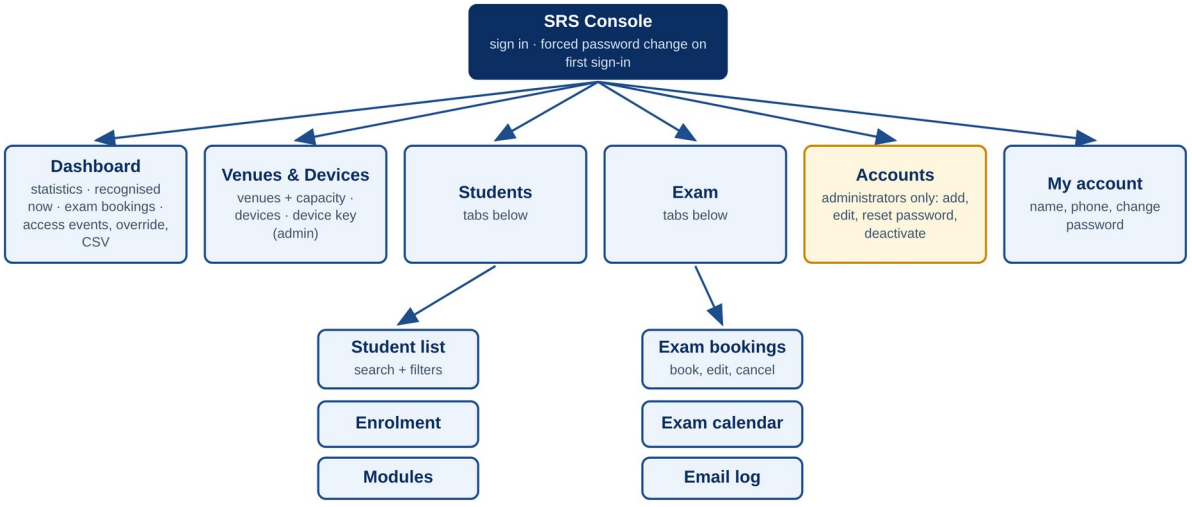


Figure 10: SRS Console menu map

How to read this figure. The top box is the console after sign-in; the second row is the left-hand menu; the bottom boxes are the tabs inside Students and Exam. The yellow Accounts page is visible to administrators only.

5.2 Conventions

Table 21: Conventions

Convention	Meaning
WARNING	Risk of injury, damage or a privacy breach
CAUTION	Risk of losing data or getting a wrong result
`monospace`	Something you type, or a value stored by the system
Bold menu names	Menu → tab → button, e.g. Students → Enrolment → Start enrolment
Decision colours	Green granted · red denied · orange error · blue already inside · grey signed out

5.3 Processing procedures

5.3.1 Sign in and sign out

1. Open the web address from 4.1.1.
2. Enter your Email and Password and press **Sign in**.
3. The Dashboard appears.
4. To leave, press the sign-out arrow next to your name at the bottom of the menu; the sign-in page returns.

5.3.2 Manage venues (administrator)

1. Open **Venues & Devices**.
2. Add a venue with its name (e.g. `Exam Hall A`) and seat **capacity**; save.
3. Edit or delete a venue from its page. A venue with door units or exam bookings cannot be deleted.

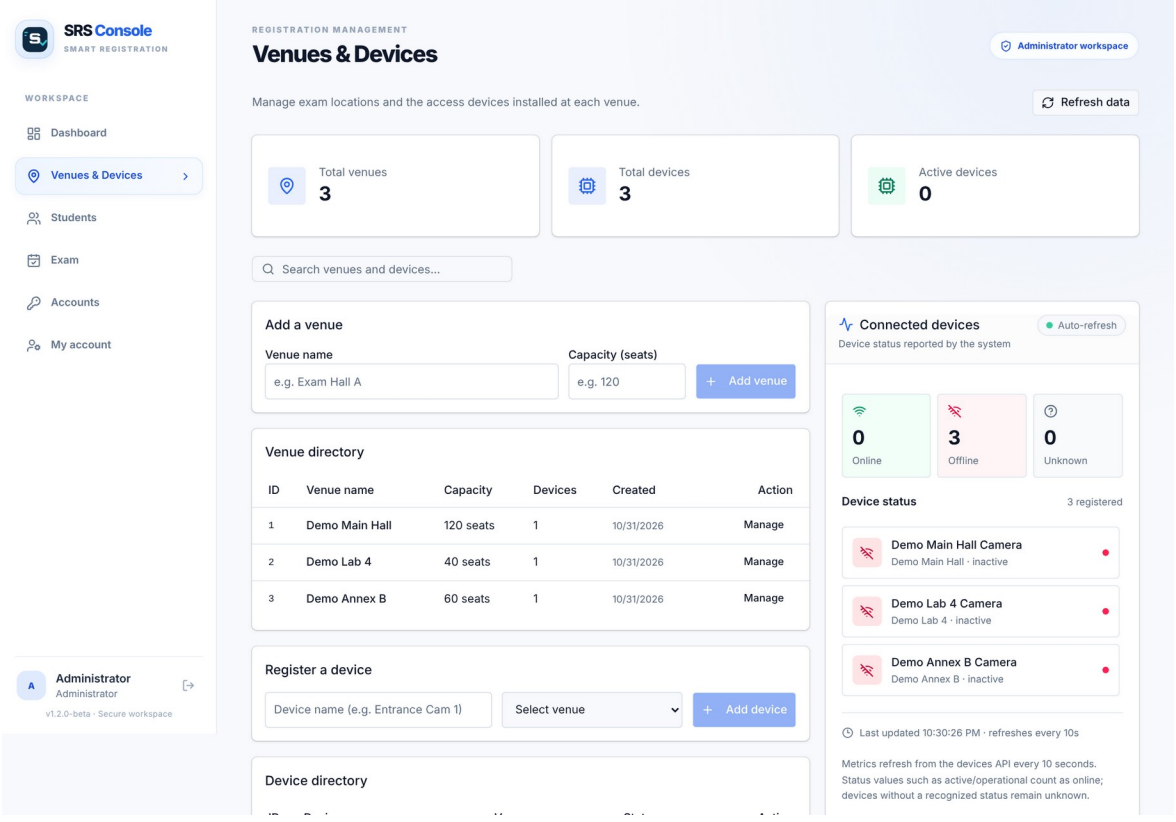


Figure 11: Venues and devices

5.3.3 Manage door units (administrator)

1. In **Venues & Devices**, add a device with a clear name (e.g. `Main Hall Door`) and its venue.
2. Open the device (**Manage**). Press **Reveal** to see the device key and **Copy** to copy it into config.ini on the door PC.
3. CAUTION: do not share the key. A device with enrolled students or recorded entries cannot be deleted. If the key may have been seen by someone else, press Replace and copy the new key into config.ini.

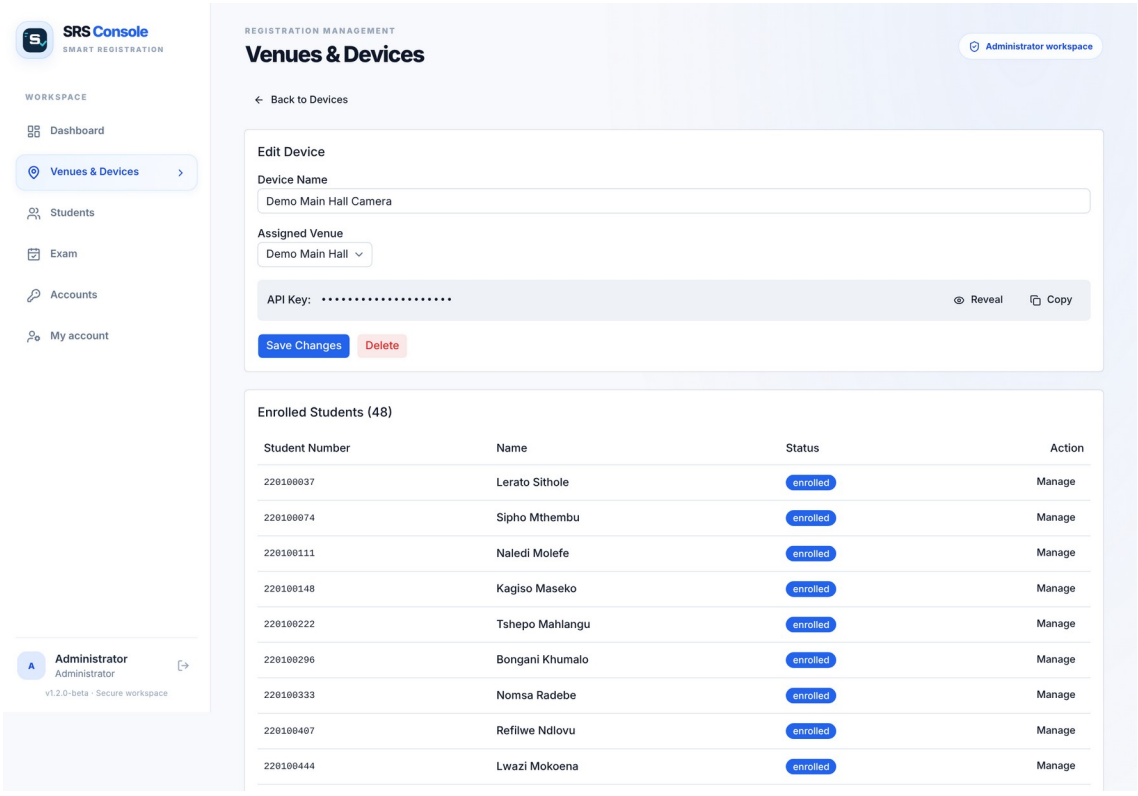


Figure 12: Device page with the hidden device key and its enrolled students

5.3.4 Enrol a student

Enrolment teaches the door unit what a student's face looks like. It is started in the console and carried out at the door unit (Figure 13).

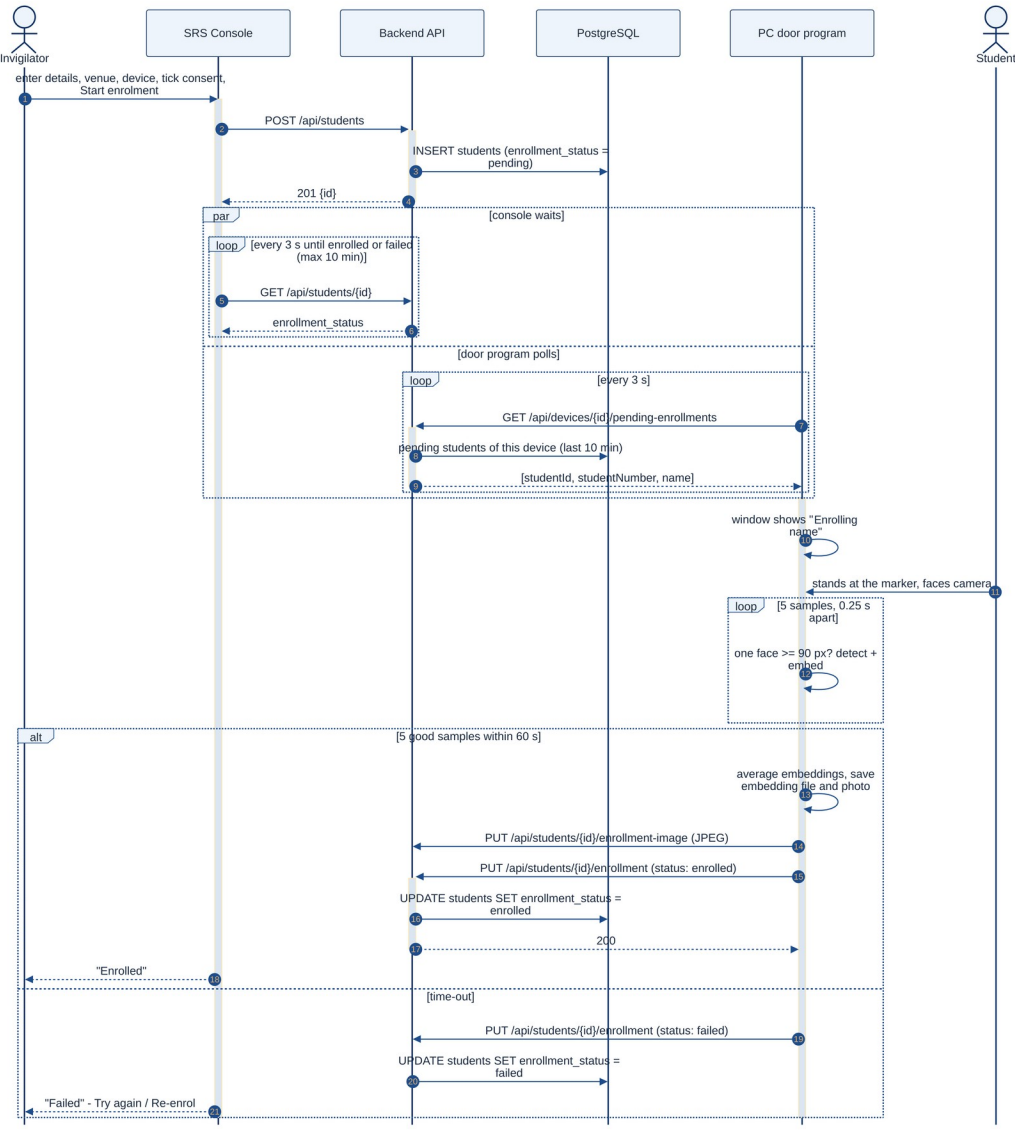


Figure 13: UML sequence diagram: enrolling a student

How to read this figure. This is a UML sequence diagram. Each participant at the top has a dashed lifeline; time runs downwards and the messages are numbered. Solid arrows are requests, dashed arrows are replies, and the thin bars show when a part is busy. Frames group messages: par = happening at the same time, loop = repeated, alt = one of the alternatives (the condition is in square brackets). The face is captured by the door unit, not by your computer.

1. Make sure the door program is running and the device shows Online.
2. Open **Students → Enrolment**; enter the student details, choose the venue and device, and tick the consent box.
3. Press Start enrolment. The page shows "Capturing…" and the door window shows "Enrolling name".
4. Ask the student to stand at the marker, face the camera and stay still. Five shots are taken automatically.

5. The page changes to **Enrolled**. If it shows **Failed**, improve the lighting, remove face coverings and press **Try again**.
6. An existing student can be enrolled again from their page (**Enrol face**) or the Recent enrolments list (Re-enrol).

CAUTION: a student is enrolled on one door unit. If the exam uses another door unit, enrol the student there.

CAUTION: one face can belong to one student only. If the face is already enrolled for another student on this door unit, the enrolment is rejected (Failed) and the door PC shows both faces side by side. If the stored face is on the wrong student, re-enrol or delete that student first.

5.3.5 Students and modules

- **Students → Student list**: search by name, number, e-mail, course or module, and filter by course, module, status or device. Open a student to edit details, modules or enrolment.
- **Students → Modules**: add a module (code and name) and register students for it. Bookings use these registrations.

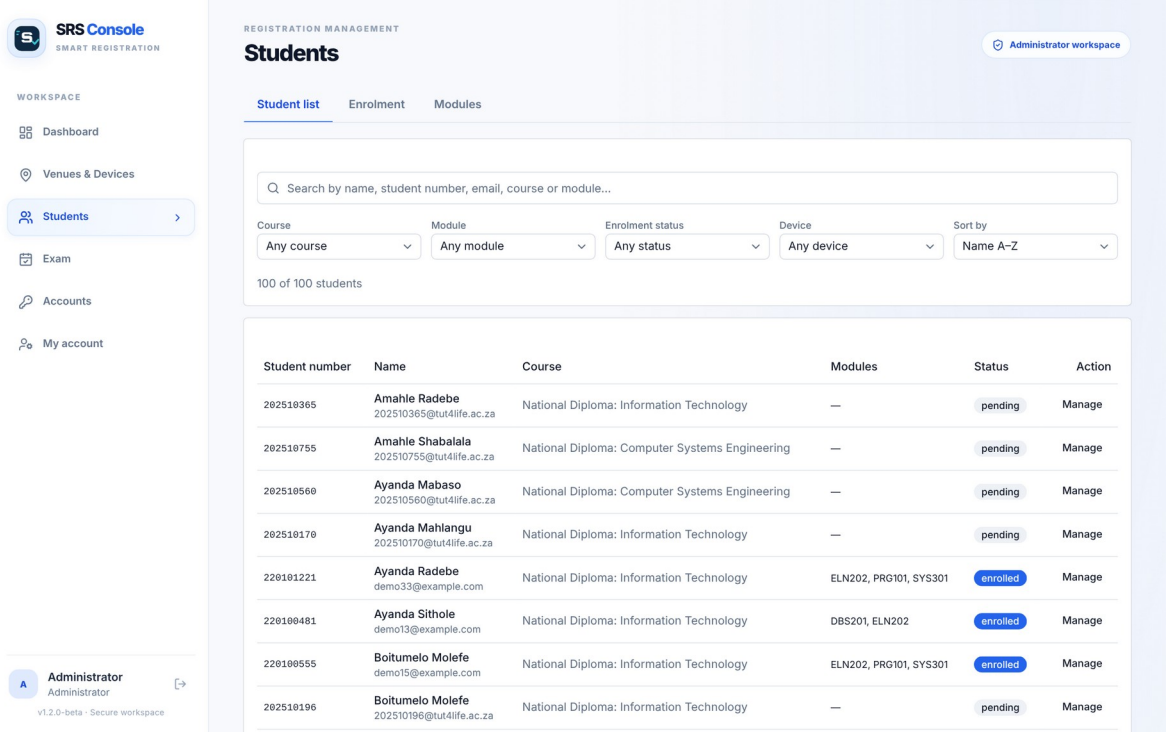


Figure 14: Student list with search and filters

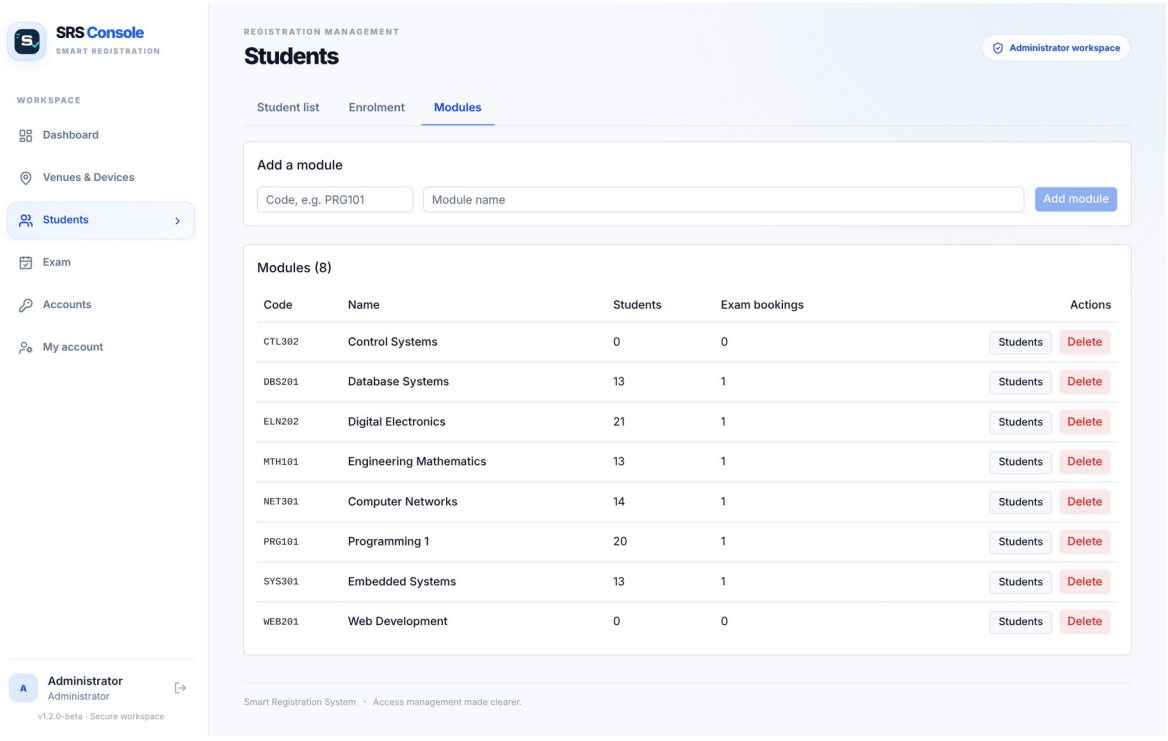


Figure 15: Modules

5.3.6 Book an exam

1. Open **Exam → Exam bookings** (or click a day in **Exam calendar**).
2. Choose the module, venue, date and session (09:00 or 13:00). The date picker shows which days still have room.
3. Save. Every student registered for the module gets a seat number in that venue and session, and an e-mail.
4. A booking is refused if the seats would exceed the venue capacity or a student already writes another exam in that session.
5. Edit or cancel a booking from the table or the calendar; the students are e-mailed about the change.

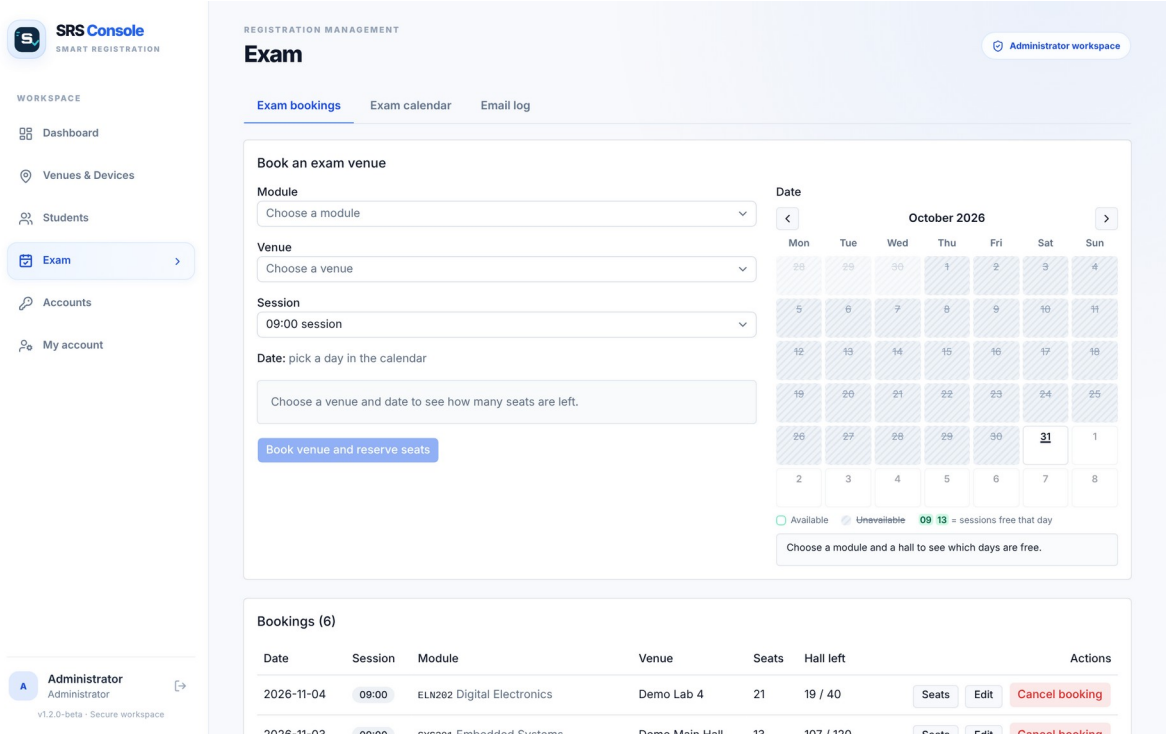


Figure 16: Exam bookings

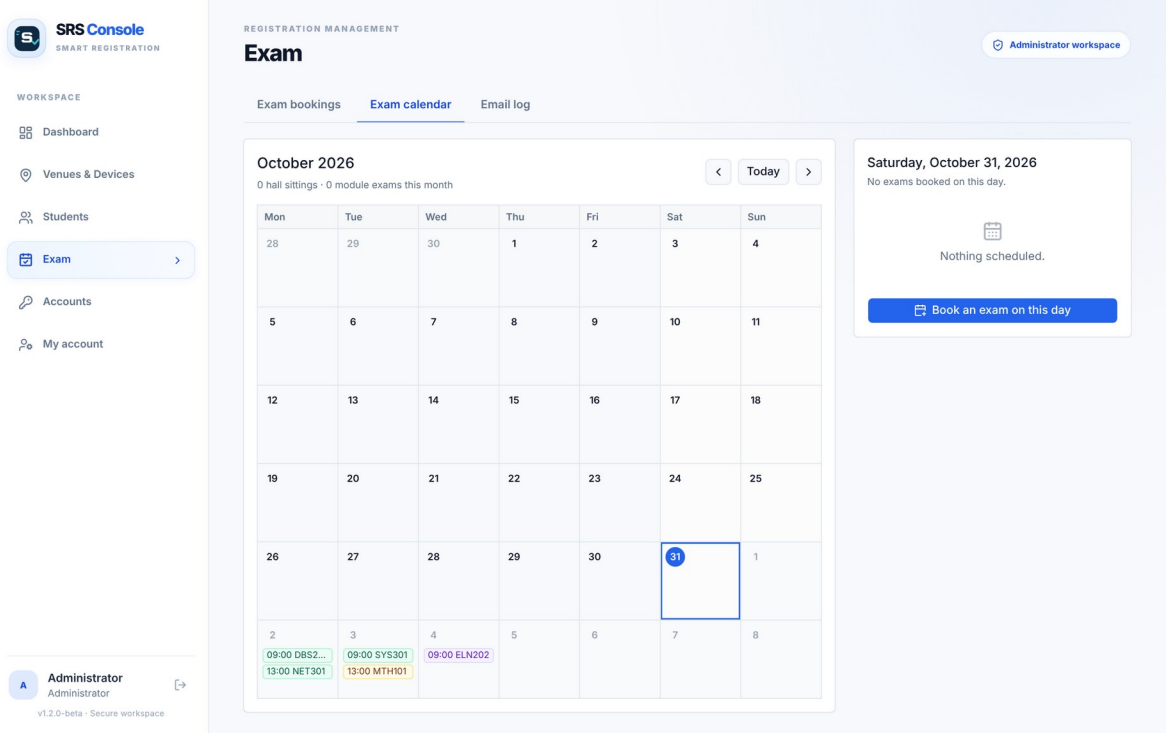


Figure 17: Exam calendar

Figure 18 shows how the two sessions of an exam day work at the door, and Figure 19 what happens inside the system when you book an exam.

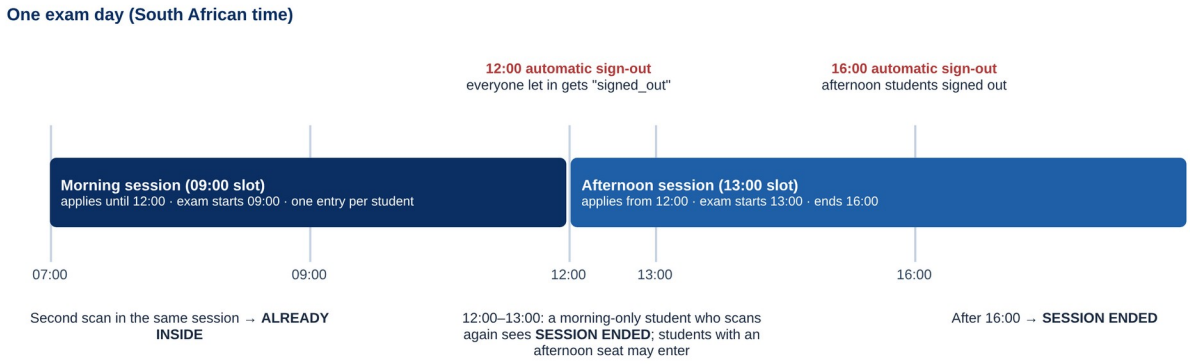


Figure 18: Exam sessions, once-per-session entry and automatic sign-out

How to read this figure. Time runs from left to right. Each coloured bar is a session; the red labels above are the automatic sign-outs; the notes below say what a student who scans again will see.

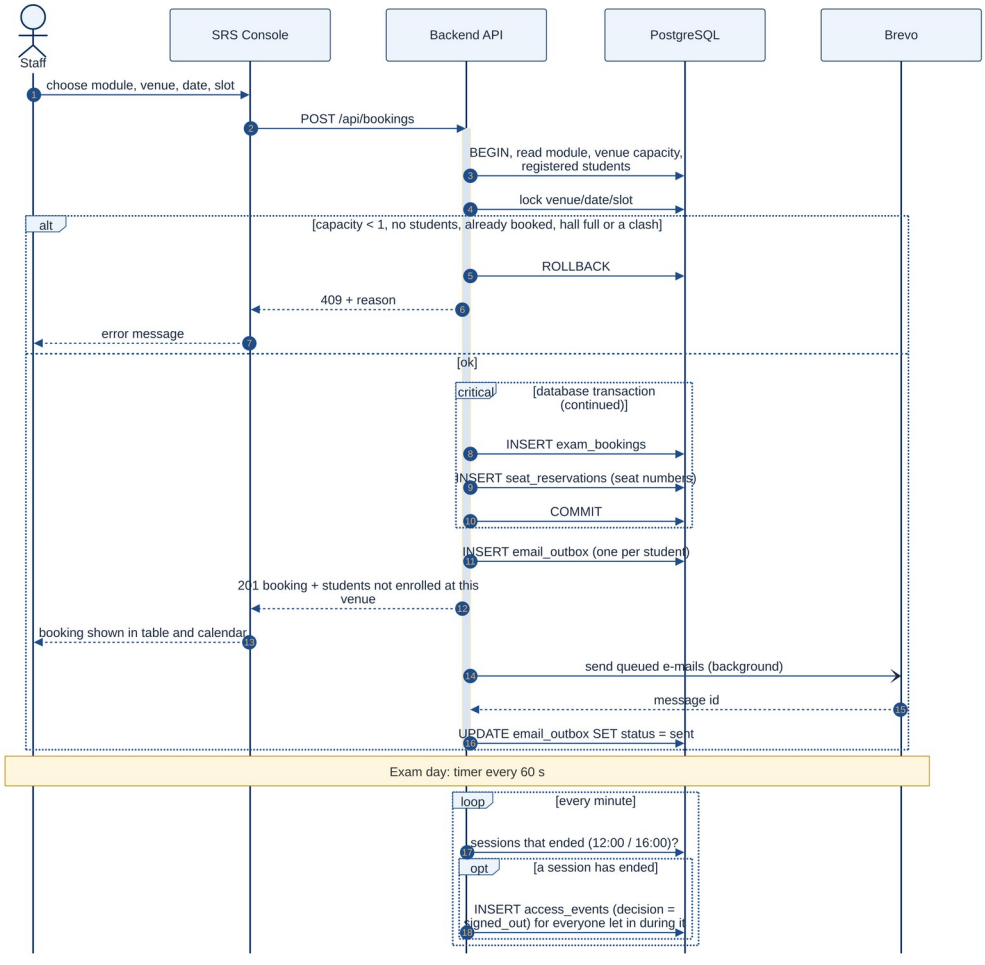


Figure 19: UML sequence diagram: booking an exam, e-mails and automatic sign-out

How to read this figure. Read the numbered messages from top to bottom. The alt frame shows the two outcomes of a booking (refused with a reason, or saved); the loop frame at the bottom is the timer that signs students out at 12:00 and 16:00.

5.3.7 Run the door unit

1. Start `python srs_pc_app.py`. Pick the webcam from the **Camera** list (press **Rescan** after plugging one in).
2. In the **Hardware** panel choose the Arduino port (e.g. `COM20 Arduino Mega 2560`) and press **Connect**. The distance turns green at 25–50 cm.
3. Press **Test hardware** once to check the Arduino, LCD, buzzer and distance sensor step by step (✔️ / ✖️ with what to check).
4. Press ▶️ **Start recognition**. The banner says "Stand at the marker and hold still for 1 second".
5. Each result appears in the banner and on the LCD (Table 18 in 4.4.2) and is added to the activity log.

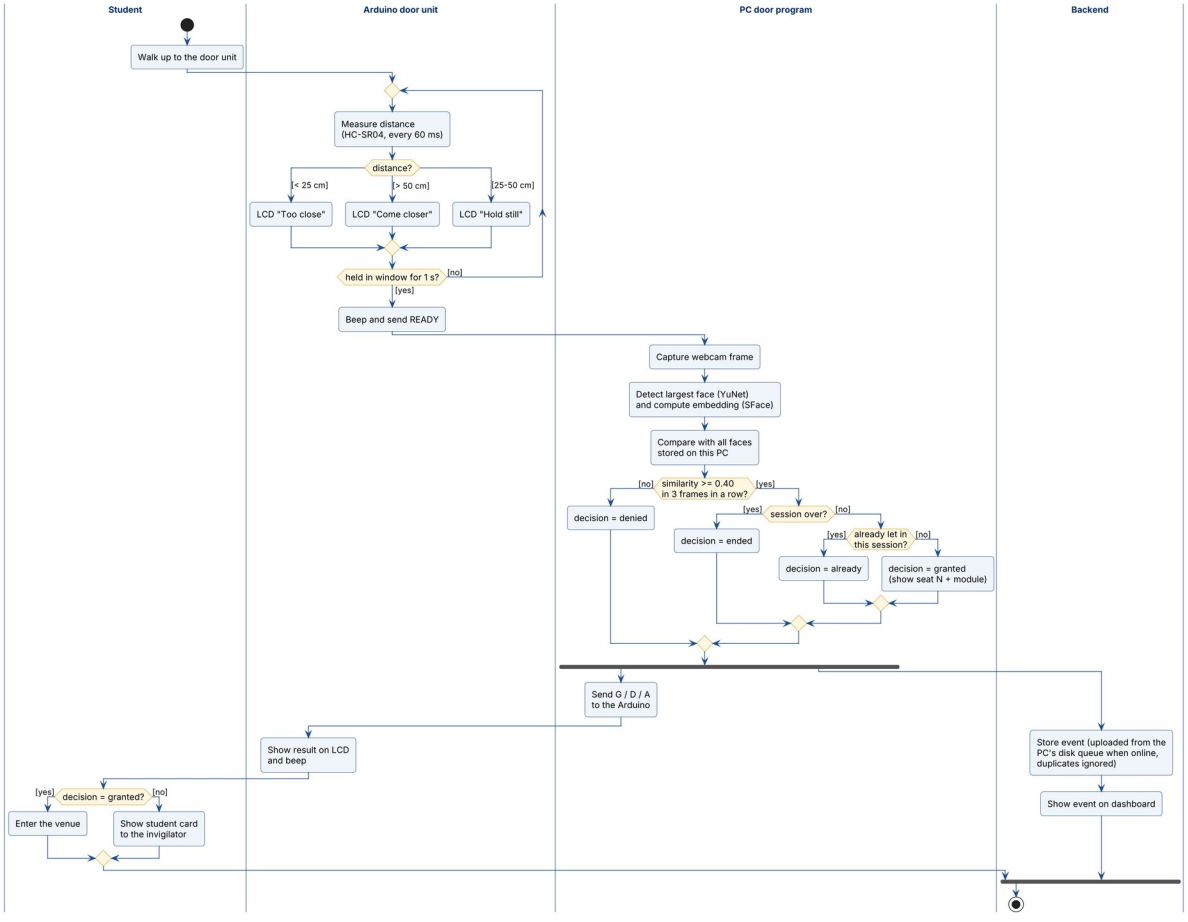


Figure 20: UML activity diagram: what happens when a student walks up

How to read this figure. This is a UML activity diagram with one swimlane (column) per part. Start at the black dot. Rounded boxes are actions; diamonds are decisions, with the condition for each path in square brackets. The thick bar splits the flow into two paths that run at the same time (the LCD result and the upload) and the second bar joins them. The ringed dot is the end.

5.3.8 Handle a denied student or a "No seat" flag (manual override)

The system is a helper. You make the final decision.

1. Ask for the student card and, if required, proof of registration. Compare the photo and details with the student.
2. If the student is genuine, admit them. If not, follow your institution's rules for suspected impersonation.
3. On the Dashboard, find the Denied row and press Override; enter the reason (your sign-in e-mail is recorded automatically). The override is stored with the event.
4. A student flagged "No seat" was recognised but has no seat in this venue and session: check their timetable before admitting them.
5. Report repeated false denials to the administrator so the student can be re-enrolled.
6. Before the exam you can print a paper sign-in sheet as a fallback: Exam → Exam bookings → Seats → Print sign-in sheet.

5.3.9 Accounts (administrator)

- Open **Accounts** to add a staff member (name, e-mail, role, optional staff number and phone). A temporary password is shown once: hand it over securely.
- Edit the role, deactivate an account, or press **Reset password** to issue a new temporary password.
- Every user can change their own name, phone and password under **My account**.

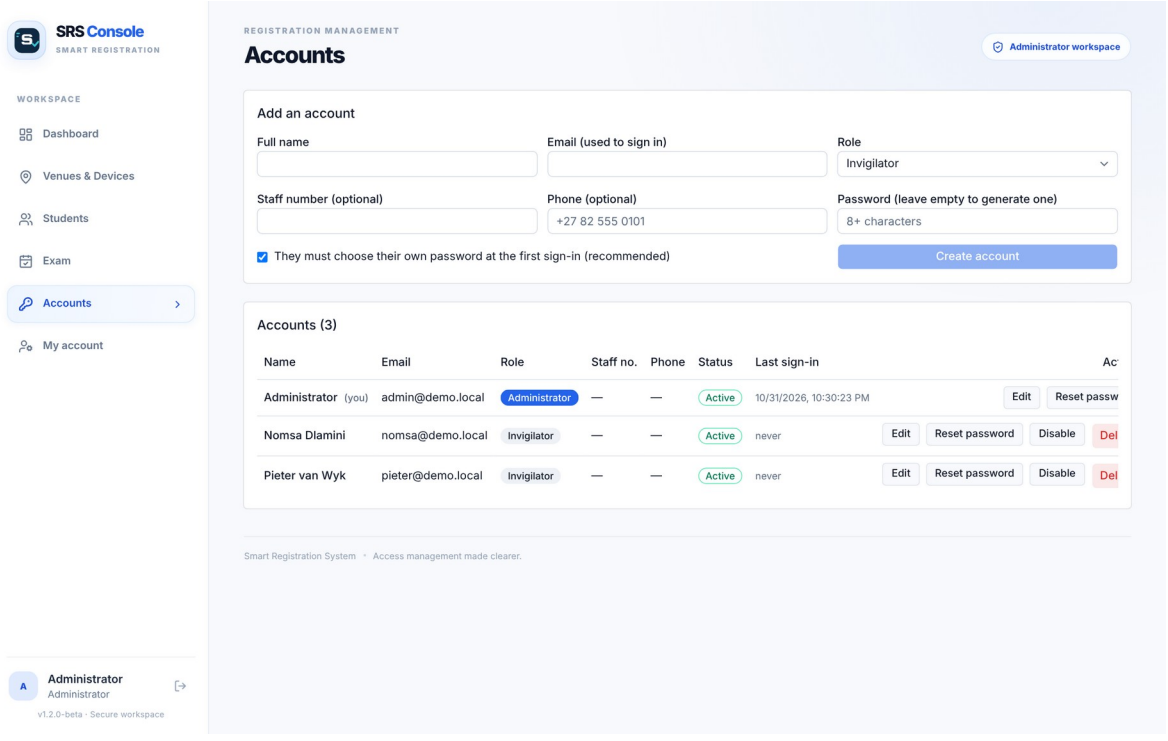


Figure 21: Accounts (administrators only)

5.3.10 E-mail log

Exam → Email log lists every e-mail with its status: sent, queued, failed, logged (no e-mail service set up) or skipped (test mode). Failed e-mails can be retried.

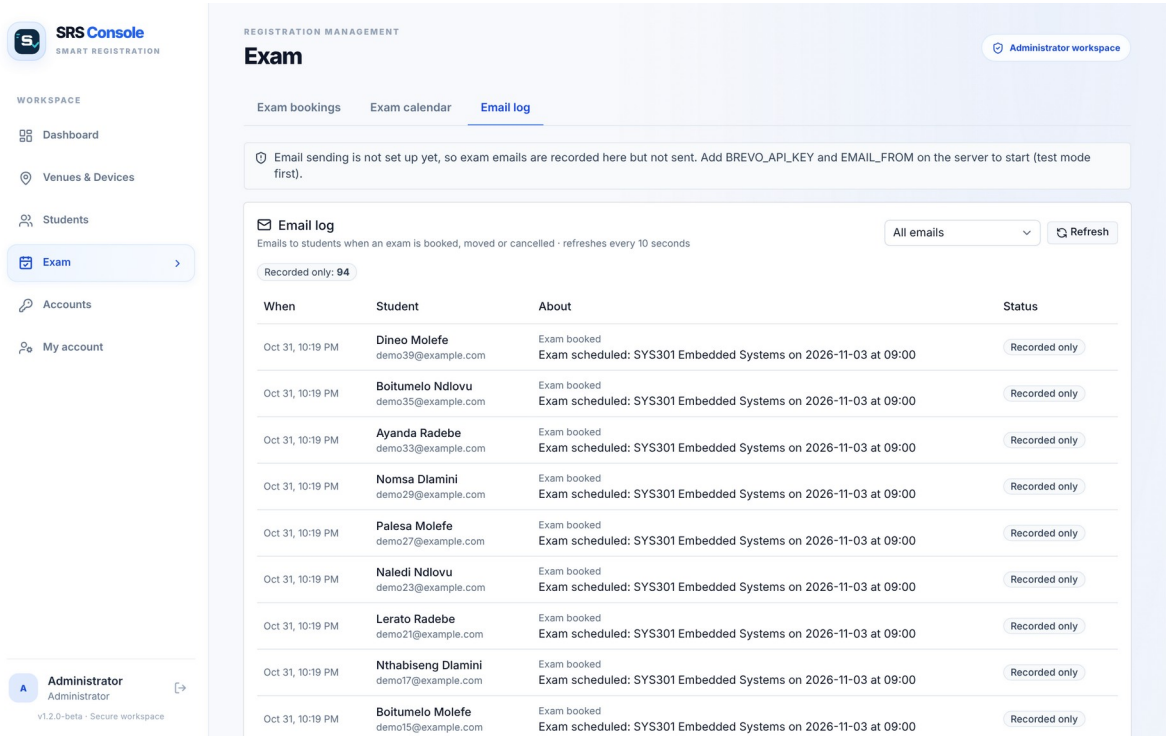


Figure 22: Email log

5.3.11 Replace a door PC (administrator)

1. Copy the folder `pc_device/data` from the old PC if possible: it holds the embeddings and photos.
2. Otherwise set up the new PC (4.1.2 B4) with the same device id and key (or press Replace on the device page and use the new key), then re-enrol the students (5.3.4).
3. Test with a known student before the exam.

5.4 Related processing (background tasks)

Table 22: Background tasks

Task	What it does	What you must do
Distance gating	The Arduino measures about 16 times a second and only asks for a picture after a 1 s hold	Stand at the marker
Local matching	The PC compares the face with every face stored on it, without the network	Nothing
Session sync	The PC downloads the seat list and who is already inside every 2 minutes and at 12:00 / 16:00	Keep the PC online
Event queue	Decisions made offline are saved to disk and uploaded in batches, never twice	Avoid closing the program while offline
Roster clean-up	Faces of students no longer enrolled on this door unit are deleted every 5 minutes	Nothing
Heartbeat	The PC tells the backend it is alive, so the console shows Online	Nothing
Automatic sign-out	At 12:00 and 16:00 everyone let in gets a signed_out event	Nothing
E-mail outbox	Booking e-mails are sent in the background; bookings never wait	Check the Email log for failures

5.5 Data backup

Backups protect the records if the database is damaged or deleted. Backup files contain personal data, so store them securely and delete them when the retention period ends.

```
# create a compressed backup of the whole database (needs the PostgreSQL client tools)
pg_dump "$DATABASE_URL" -Fc -f srs_backup.dump
# restore that backup into an empty database (--clean removes existing objects first)
pg_restore --clean --no-owner -d "$DATABASE_URL" srs_backup.dump
```

Neon also offers its own restore points, depending on the plan. On each door PC, copy `pc_device/data` and `config.ini` to a secure location after enrolment. Keep an offline copy of the Render settings.

5.6 Recovery from errors, malfunctions and emergencies

- **Problem during admission:** use manual card checks; do not delay students. Fix the SRS afterwards.
- **Door program crashed:** start it again. Faces and the queue are on disk and are kept.
- **Backend or database problem:** restore from backup (5.5), then confirm with a test scan.
- **Emergency evacuation:** leave the venue first; the equipment can be replaced.

5.7 Messages

Table 23: Messages

Source	Message	Type	Meaning / action
Console	incorrect email or password	Error	Retype; after 5 failures wait 15 minutes
Console	new password must be at least 8 characters	Error	Choose a longer password
Console	only an administrator can do that	Error	Ask an administrator
Console	Cannot delete venue: … device(s) still assigned	Error	Move or delete the devices first
Console	Capturing… / Enrolled / Failed	Information	Enrolment progress (5.3.4)
Console	Entry override recorded for event N	Information	Override saved
LCD	Smart Register / Step up to scan	Information	Waiting for a student
LCD	NOT CONNECTED / Start the PC app	Error	The door program is not running: start it and press Connect (5.3.7)
LCD	Too close / Step back · Come closer / Stand at 25-50cm	Instruction	Move to the marker
LCD	Hold still 32cm / ######	Information	Counting the 1-second hold
LCD	Scanning face... / Look at camera	Information	Picture being taken
LCD	ACCESS GRANTED · ACCESS DENIED · ALREADY INSIDE	Result	See Table 18
LCD	No answer / Please try again	Error	Step up again
LCD	Next student / Step away first	Instruction	The previous student must step back
PC window	No camera found. Plug in the webcam and press Rescan.	Error	Connect the webcam
PC window	Recognition is off – press Start recognition to scan students	Information	Press ▶️ Start recognition

5.8 Quick-reference guide

Quick-Start Card (print on one page)

Before the exam: exam booked · every student "enrolled" on this venue's door unit.

At the venue: PC on mains · webcam at face height · Arduino plugged in · marker 25–

50 cm from the sensor · start `python srs_pc_app.py` · Hardware → Connect · ▶ Start recognition · sign in to the console · device Online · test with yourself.

During admission: one student at a time at the marker, hold still 1 s. **ACCESS GRANTED** → admit (check the card if the dashboard says "No seat"). **ACCESS DENIED** → check the card yourself, then **Override** if genuine.

If something breaks: switch to manual checks straight away; LCD blank → Test hardware; "No camera" → Rescan; Offline → check internet, wait a minute.

After the exam: sign-out is automatic at 12:00 / 16:00 · ■ Stop recognition · close the window · sign out · Export CSV if needed.

6. Query Procedures

A query is a request to look up records. Invigilators look up records in the console. Administrators can also query the database directly.

6.1 Database and data file format

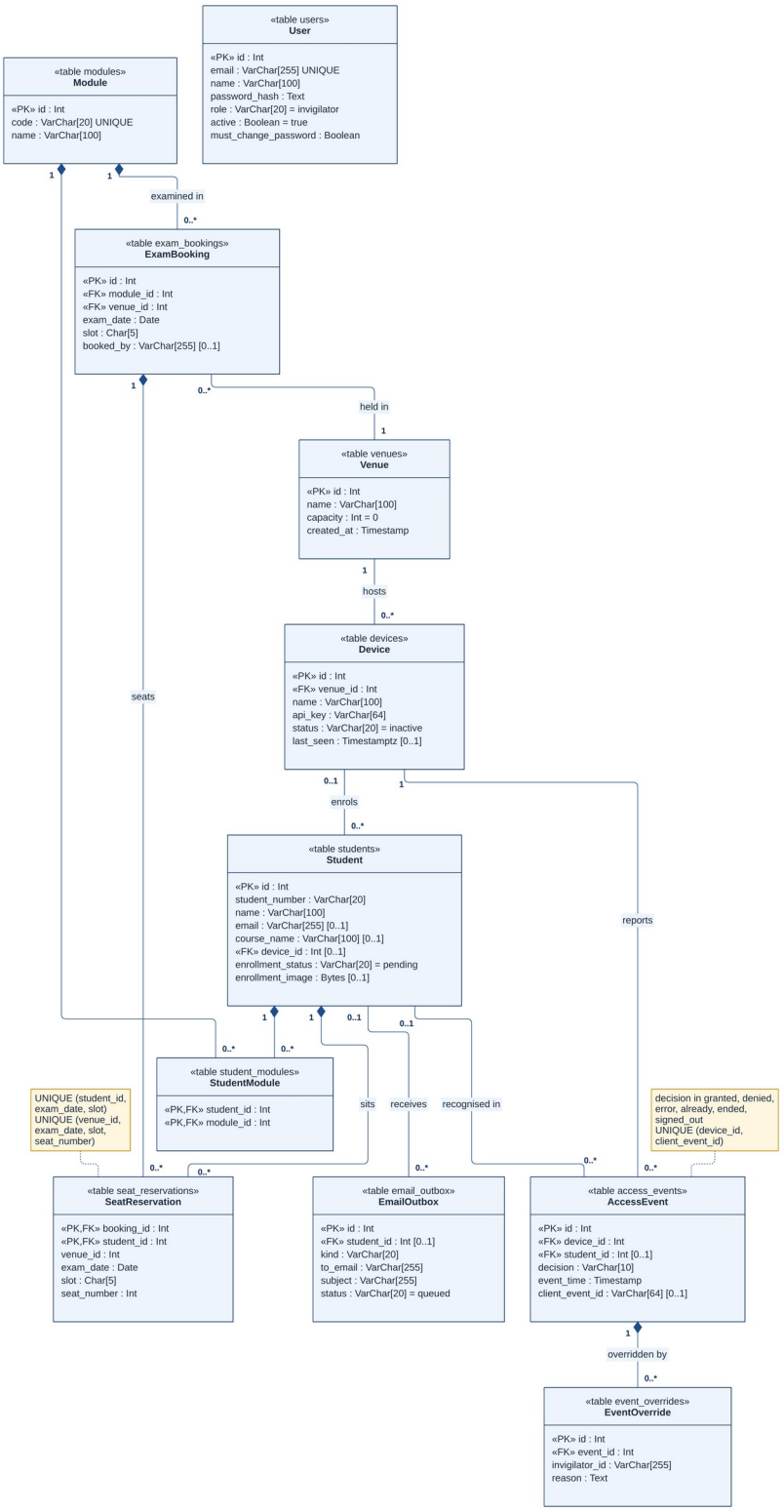


Figure 23: UML class diagram of the database (11 tables)

How to read this figure. This is a UML class diagram: each box is one table (its name on top, its columns and data types below). «PK» marks the unique id and «FK» a column that

points to another table. The numbers at each end of a line are multiplicities: 1 = exactly one, 0..1 = none or one, 0..* = any number. A filled diamond means the rows on the other end are deleted together with their owner.

Table 24: Main tables

Table	Holds	Key columns
venues	Exam venues	id, name, capacity
devices	Door units	id, venue_id, name, api_key (secret), status, last_seen
students	Students	id, student_number, name, email, course_name, device_id, enrollment_status, enrollment_image
modules / student_modules	Modules and who is registered	code, name · student_id, module_id
exam_bookings	Booked exams	module_id, venue_id, exam_date, slot (09:00 / 13:00)
seat_reservations	Seats	booking_id, student_id, venue_id, exam_date, slot, seat_number
access_events	Every decision	device_id, student_id (empty = unknown), decision, event_time, client_event_id
event_overrides	Manual decisions	event_id, invigilator_id, reason, created_at
email_outbox	E-mails	kind, to_email, subject, status, sent_at
users	Staff accounts	email, name, role, active, must_change_password, last_login_at

6.2 Query capabilities

- **Dashboard → Recent access events:** search by student, device or decision; filter by decision and date; **Export CSV**.
- **Students → Student list:** search and filter by course, module, enrolment status and device.
- **Exam calendar / Exam bookings:** bookings by day, venue and session, with seats booked.
- **Database (administrators):** any query with psql or the Neon SQL editor (6.3).

6.3 Query preparation

```
# open an interactive session on the database
psql "$DATABASE_URL"
```

Example 1: every denied attempt today, with the door unit.

```
-- denied attempts today (times are stored in UTC, so convert them to South African time first)
SELECT e.event_time, d.name AS device
FROM access_events e JOIN devices d ON d.id = e.device_id
WHERE e.decision = 'denied' AND (e.event_time AT TIME ZONE 'UTC' AT TIME ZONE 'Africa/Johannesburg')::date = (now() AT TIME ZONE 'Africa/Johannesburg')::date
ORDER BY e.event_time;
```

Example 2: attendance for one exam (seated students and their first entry).

```
-- replace the module code, date and slot with your own
SELECT s.student_number, s.name, r.seat_number, MIN(e.event_time) AS first_entry
FROM seat_reservations r
JOIN exam_bookings b ON b.id = r.booking_id
JOIN modules m ON m.id = b.module_id
JOIN students s ON s.id = r.student_id
```

```
LEFT JOIN access_events e ON e.student_id = s.id AND e.decision = 'granted'
  AND (e.event_time AT TIME ZONE 'UTC' AT TIME ZONE 'Africa/Johannesburg')::date =
DATE '2026-11-02'
WHERE m.code = 'PRG101' AND b.exam_date = DATE '2026-11-02' AND b.slot = '09:00'
GROUP BY s.student_number, s.name, r.seat_number ORDER BY r.seat_number;
```

Example 3: students per enrolment status and venue.

```
SELECT v.name AS venue, s.enrollment_status, COUNT(*) AS students
FROM students s
JOIN devices d ON d.id = s.device_id JOIN venues v ON v.id = d.venue_id
GROUP BY v.name, s.enrollment_status ORDER BY v.name, s.enrollment_status;
```

Example 4: overrides with the reason.

```
SELECT o.created_at, o.invigilator_id, o.reason, e.device_id
FROM event_overrides o JOIN access_events e ON e.id = o.event_id
ORDER BY o.created_at DESC;
```

6.4 Control instructions

1. Run queries only from a trusted computer.
2. Use SELECT for reading. Do not run UPDATE or DELETE on live data without a backup (5.5).
3. Close the session with `\q`.
4. Handle results as confidential personal information.

7. Notes

7.1 Acronyms and abbreviations

Table 25: Acronyms and abbreviations

Term	Meaning
API	Application Programming Interface: the web addresses one program offers to another
Arduino	A small microcontroller board; here it runs the distance sensor, LCD and buzzer
CSV	Comma-Separated Values: a plain spreadsheet file
Embedding	A list of 128 numbers representing a face, used to compare faces
FAR / FRR	False Acceptance Rate (stranger accepted) / False Rejection Rate (enrolled student refused)
HC-SR04	An ultrasonic sensor that measures distance by timing a sound echo
HTTPS	The secure request/response protocol used between all parts of the system
I²C	A two-wire connection used for the LCD
JWT	JSON Web Token: the signed sign-in token the console sends with every request
LCD	Liquid Crystal Display: the 16×2 character screen on the door unit
OpenCV	Open-source computer vision library that runs the face models
POPIA	Protection of Personal Information Act 4 of 2013
SFace / YuNet	The face recognition model / face detection model
SRS	Smart Registration System
TUT	Tshwane University of Technology

7.2 Known limitations

- Not connected to the university student information system.

- Accuracy depends on lighting, distance and camera quality; always keep manual checks available.
- No liveness check: a good printed photo of an enrolled student could be accepted. The invigilator must watch the door.
- A recognised student without a seat is let in and only flagged; the invigilator decides.
- Free hosting tiers sleep when idle; the first request can take up to a minute.

7.3 Document history

Table 26: Document history

Version	Date	Change
1.0	08/10/2026	First full manual (ESP32-CAM design)
2.0	05/11/2026	Rewritten for the final system: PC + webcam + Arduino door unit, staff accounts, modules, bookings, seats, e-mails, automatic sign-out; measured performance; new figures

7.4 Summary

This manual describes how to set up and use the Smart Registration System at an exam venue. A door unit at the entrance checks a student's face once the student holds still 25–50 cm from it, lets each recognised student in once per session, shows the seat on the PC and the result on an LCD, and reports every decision to the SRS Console. Invigilators enrol students, book exams, watch the live dashboard and make the final decision on any denied student. Administrators deploy the backend and console, set up door units, manage accounts and back up the database. The system handles biometric data, so consent, secure handling and a manual fallback are essential. The most important rule on exam day: admission never waits for the system. If anything fails, check student cards manually.

7.5 Figure sources

All diagrams are the author's own. The UML sequence, class and state machine diagrams were drawn in Mermaid; the UML use case, deployment and activity diagrams in PlantUML (Mermaid has no UML form for these); the other diagrams were drawn as web pages. All screenshots are of the SRS Console running with demonstration data. No figure is taken from another source. The diagram source files are in docs_out/figures/mermaid/src and docs_out/figures/uml/src.

8. References

Du, C. and Muwanguzi Ntume, M. (2025) Project Design 376 / Industry Exposure Student Course Guide (PD2316D). Pretoria: Tshwane University of Technology.

Arduino (n.d.) Arduino Mega 2560 Rev3. Available at: https://docs.arduino.cc/hardware/mega-2560/ (Accessed: 31 October 2026).

Information Regulator (South Africa) (2021) Guidance Note on Processing of Special Personal Information. Available at: https://inforegulator.org.za (Accessed: 31 October 2026).

OpenCV (n.d.) OpenCV Zoo: face_detection_yunet and face_recognition_sface. Available at: https://github.com/opencv/opencv_zoo (Accessed: 31 October 2026).

PostgreSQL Global Development Group (2024) PostgreSQL 16 Documentation. Available at: https://www.postgresql.org/docs/16/ (Accessed: 31 October 2026).

Prisma Data (n.d.) Prisma ORM Documentation. Available at: https://www.prisma.io/docs (Accessed: 31 October 2026).

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

Wu, W., Peng, H. and Yu, S. (2023) 'YuNet: a tiny millisecond-level face detector', Machine Intelligence Research, 20(5), pp. 656–665.

Zhong, Y. et al. (2021) 'SFace: sigmoid-constrained hypersphere loss for robust face recognition', IEEE Transactions on Image Processing, 30, pp. 2587–2598.

Appendix A: Hardware Guide

A.1 Parts list

Table 27: Door unit parts

Part	Purpose
Windows PC or laptop	Runs the door program and face recognition
USB webcam	Takes the pictures
Arduino Mega 2560 (or Uno/Nano) + USB cable	Runs the distance sensor, LCD and buzzer; powered by the PC
HC-SR04 ultrasonic sensor	Measures the distance to the student
16×2 LCD with I²C backpack	Shows instructions and the result
Buzzer (passive or active)	Beeps the result
Breadboard, jumper wires, enclosure / stand	Holds the parts neatly at the entrance

A.2 Wiring

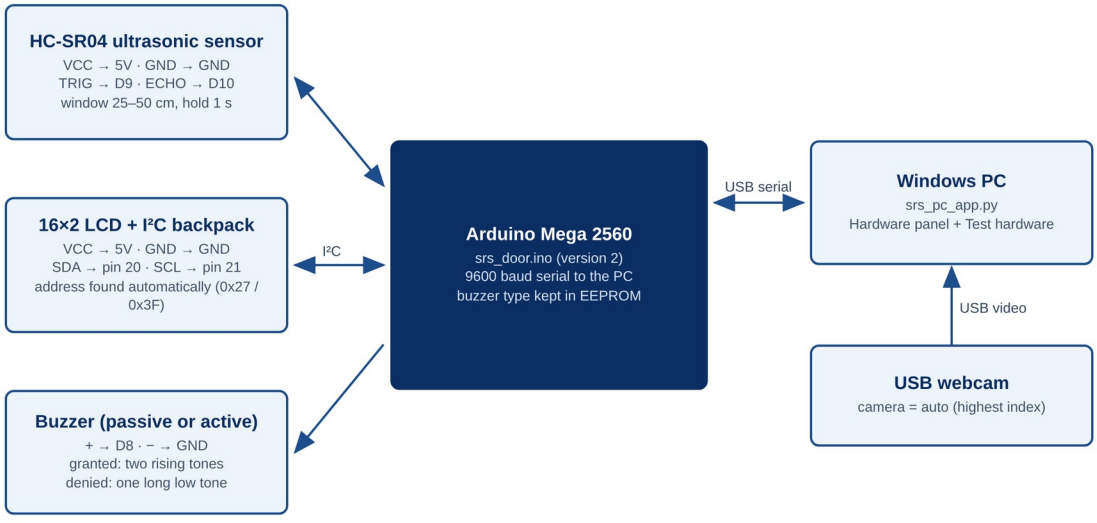


Figure 24: Door unit wiring

How to read this figure. The dark box is the Arduino. Each part is listed with the Arduino pin each of its wires goes to; arrows show the direction of the signal, and double arrows mean data flows both ways.

Table 28: Wiring (Arduino Mega)

Part	Wire	Arduino pin
HC-SR04	VCC / GND / TRIG / ECHO	5V / GND / D9 / D10
LCD I²C backpack	VCC / GND / SDA / SCL	5V / GND / pin 20 / pin 21 (Uno/Nano: A4 /

Part	Wire	Arduino pin
		A5)
Buzzer	+ / −	D8 / GND
PC	USB	USB port (power and serial, 9600 baud)

A.3 Power-up and shutdown

1. Plug the Arduino into the PC; the LCD shows "Smart Register / Starting..." and then "NOT CONNECTED / Start the PC app" until the door program connects.
2. Start the door program and press Connect in the Hardware panel. The LCD changes to "Smart Register / Step up to scan".
3. Shutdown: close the door program first, then unplug the Arduino.
4. CAUTION: do not move wires while the Arduino is powered.

A.4 Mounting and positioning

- Mount the webcam at face height, facing the place where students will stand, with the HC-SR04 next to it.
- Put a floor marker so the student's face is 25–50 cm from the sensor (about 35 cm works well).
- Keep the space between the sensor and the student clear: the HC-SR04 measures the nearest object in a narrow cone.
- Avoid strong light behind the student (windows, open doors). Even, front-facing light works best.
- Keep cables tidy and out of walkways to avoid trip hazards.

A.5 Hardware test

Test hardware in the door program checks, one by one: (1) Arduino connected over USB, (2) door unit program answers (version 3), (3) LCD found on the I²C bus, (4) LCD shows the test screen (you answer Yes/No), (5) buzzer beeps (if not, it tries the other buzzer type and the Arduino remembers it), (6) distance sensor gives readings, (7) it sees your hand at 25–50 cm. Each step is marked ✔️ or ✖️ with what to check.

A.6 Care and maintenance

- Wipe the webcam lens with a soft dry cloth.
- Keep the door unit away from water and heat.
- If the LCD is blank, turn the blue contrast screw on its back.

Appendix B: Door Program Settings

Settings live in `pc_device/config.ini` (kept out of Git because it holds the device key). Each can also be given on the command line.

Table 29: config.ini and command-line options

config.ini	Command line	Meaning	Default
url	--url	Backend address	https://smart-registration-system.onrender.com
device_id	--device-id	The device id shown in the console	–
device_key	--key	The device key (administrators: Reveal on the device page)	–
camera	--camera	Webcam number, or auto (highest number)	auto

config.ini	Command line	Meaning	Default
serial_port	--serial	Arduino port, e.g. COM3 (empty without an Arduino)	–
min_face_px	--min-face	Faces narrower than this are "too far away"	90
–	--check	Test the server, key and models, then exit (no camera)	–

Matching uses SFace cosine similarity with threshold 0.40 (`MATCH_THRESHOLD` in `face_engine.py`). Raise it if strangers are ever accepted; lower it if enrolled students are often denied.

Appendix C: Database Creation

The database is created and updated automatically: `npm start` in the Backend folder runs `node prisma/deploy.js`, which applies every migration in `Backend/prisma/migrations` before the server starts. The schema is defined in `Backend/prisma/schema.prisma` (Figure 23). To create demonstration data on a local test database only:

```
# start the backend against a local test database (applies the migrations)
npm start
# in a second terminal: 3 venues, 3 devices, 6 modules, 40 students, 6 bookings, 27 events
SEED_EMAIL=<admin e-mail> SEED_PASSWORD=<password> npm run seed-demo
# optional: more students and exam bookings for the next two weeks
SEED_EMAIL=<admin e-mail> SEED_PASSWORD=<password> npm run seed-exams
# optional: sample access events for the last 7 days (for the dashboard chart)
SEED_EMAIL=<admin e-mail> SEED_PASSWORD=<password> npm run seed-activity
```

CAUTION: the seed scripts refuse to run against anything except localhost unless `CONFIRM_SEED=yes` is set. Never run them against the live database.